Information Sciences 509 (2020) 320–342

Contents lists available at ScienceDirect

Information Sciences

journal homepage: www.elsevier.com/locate/ins

Fast hypervolume approximation scheme based on a segmentation strategy☆

Weisen Tang[a], Hai-Lin Liu[a,*], Lei Chen[a], Kay Chen Tan[b], Yiu-ming Cheung[c]

[a] Guangdong University of Technology, Guangzhou, China
[b] Department of Computer Science, City University of Hong Kong, Hong Kong SAR, China
[c] Department of Computer Science, Hong Kong Baptist University, Hong Kong SAR, China

ARTICLE INFO

Article history:
Received 22 January 2018
Revised 15 February 2019
Accepted 20 February 2019
Available online 27 February 2019

Keywords:
Many-objective
Hypervolume
Evolutionary algorithm
Monte carlo simulation

ABSTRACT

Hypervolume indicator based evolutionary algorithms have been reported to be very promising in many-objective optimization, but the high computational complexity of hypervolume calculation in high dimensions restrains its further applications and developments. In this paper, we develop a fast hypervolume approximation method with both improved speed and accuracy than the previous approximation methods via a new segmentation strategy. The proposed approach consists of two crucial process: segmentation and approximation. The segmentation process recursively finds areas easy to be measured and quantified from the original geometric figure as many as possible, and then divides the measurement of the rest areas into several subproblems. In the approximation process, an improved Monte Carlo simulation is developed to estimate these subproblems. Those two processes are mutually complementary to simultaneously improve the accuracy and the speed of hypervolume approximation. To validate its effectiveness, experimental studies on four widely-used instances are conducted and the simulation results show that the proposed method is ten times faster than other comparison algorithms with a same measurement error. Furthermore, we integrate an incremental version of this method into the framework of SMS-EMOA, and the performance of the integrated algorithm is also very competitive among the experimental algorithms.

© 2019 Elsevier Inc. All rights reserved.

1. Introduction

Multiobjective optimization problems (MOPs) often involve multiple conflicting objectives to be optimized simultaneously in real-world applications. Due to the conflicting nature of the objectives, MOPs do not exist one single solution that is able to optimize all the objectives. Instead, they aim at finding a series of best possible trade-off solutions termed Pareto optimal solutions, whose objectives cannot be improved further more. The Pareto optimal solutions are known as the Pareto front (PF) in objective space and the Pareto set (PS) in decision space, respectively. Evolutionary multiobjective optimization (EMO) algorithms, as a class of population based search heuristics, have been successfully applied in various bi-objective and tri-objective optimization scenarios. Based on various acceptance rules of selecting offspring solutions, EMO algorithms

☆ This paper belongs to the special issue "Many Objective Opt" edited by Prof. W. Pedrycz.

* Corresponding author.
E-mail addresses: hlliu@gdut.edu.cn, lhl@scnu.edu.cn (H.-L. Liu).
URL: http://www.lhl-gdut.cn/lhl/ (H.-L. Liu)

https://doi.org/10.1016/j.ins.2019.02.054
0020-0255/© 2019 Elsevier Inc. All rights reserved.

could be generally divided to three branches: 1) Pareto dominance-based approaches, e.g. see [16], 2) Decomposition-based methods, e.g. see [12,22,23,30,31,46], and 3) Indicator-based algorithms, e.g. [5,48]. Pareto dominance-based algorithms rely on the relationship of Pareto dominance among solutions to approach the PF, Decomposition-based methods decompose the original problem into several subproblems and utilize scalarizing function to conduct selections in each subproblem. However, they encounter various kinds of difficulties when the number of objectives increases to more than three [11], which are known as many-objective optimization problems (MaOPs) [13,41]. In contrast to the two other groups of EMO algorithms, indicator-based algorithms use performance indicators to guide the evolution (*e.g.*, hypervolume, I_{SDE}) [5,29]. Prominent examples of this kind of EMO algorithms are IBEA [48], SMS-EMOA [5,32], MO-CMA-ES [26], and R2-EMOA [42], etc. In this paper, we mainly focus on the hypervolume indicator based EMO algorithms.

Hypervolume indicator is a volumetric measurement in geometry that calculates the union volume of a dominated region which was produced by a set of points. It was originally proposed as a set quality indicator called 'size of the space covered' to quantitatively compare the populations of different EMO algorithms [5,49]. This concept was later denoted as 'hypervolume measurement' [5]. Because hypervolume indicator has a significant attribute that keeps fully sensitive and effective to both Pareto dominance and population diversity [10,20], it is now one of the most important indicators in many-objective optimization.

Hypervolume indicator based EMO algorithms such as HypE [2], SMS-EMOA [5], FV-MOEA [27], HAGA [38] and MOP-SOhv [21], etc have been reported to be very promising in solving MaOPs. For instance, S Metric Selection Evolutionary Multi-Objective Algorithm (SMS-EMOA) [5,34], is designed to maximize the exact hypervolume of populations by means of discarding the least contribution solution. This idea in essence utilizes the hypervolume indicator to measure the quality of solutions. The superiority of SMS-EMOA has been verified by a plethora of research studies [33,47]. Unfortunately, it is still unpractical to apply SMS-EMOA to high-dimensional MaOPs due to the costly computational resources of hypervolume indicator calculation [4,7,8]. In [7], Bringmann and Friedrich proved that hypervolume calculation is a NP–complete problem. It is expected that no polynomial calculation algorithm exists since this would imply $NP = P$ [4,8].

Therefore, the main problem of this kind of EMO algorithms can be attributed to find a method that calculates the hypervolume indicator (or hypervolume measurement) efficiently. Without a fast and quality hypervolume calculation algorithm, the hypervolume based EMO algorithms would either have high computational complexity or compromised performance [2,9].

A lot of methods have been proposed to calculate the hypervolume indicator, and generally, two classes can be seen. The first class is the exact calculation method, which is mainly realized by means of the recursive computation. It divides the hypervolume calculation problem into several subproblems with the same property as the original problem and solves them respectively. This kind of method is featured by high accuracy and exponentially increasing complexity. Some influential algorithms of this kind include hypervolume by slicing objectives (HSO) [6,18] algorithm, HOY [3,24,36], Quick hypervolume (QHV) [39,40], HBDA [28] and Walking Fish Group (WFG) [14,37,43,44], etc. The second class is to estimate the hypervolume value by some statistical methods [1,7]. Monte Carlo simulation is a representative method, and its effectiveness in high dimensions has been proved by some performance analyses [35]. The first attempt in this direction is presented in [1]. Bringmann and Friedrich [7] later presented an efficient FPRAS (fully polynomial-time randomized approximation scheme) algorithm for hypervolume calculation. Its complexity is $O(dn/\epsilon)$ with the error of $\pm\sqrt{\epsilon}$. Though this method can reduce the complexity of hypervolume computation, the accuracy and running time have to be compromised. The main reason is the huge number of sampling points needed to reach the condition of dependable accuracy, which seriously influences the running time. Therefore, how to get it a further performance boost is still one of the major research problems.

In this paper, we propose a new hypervolume approximation method, which consists of two parts: segmentation and approximation. The basic idea of this method is to reduce the number of Monte Carlo sampling points by a new segmentation strategy. This segmentation strategy recursively segments the original geometric problem into a hypercube and several subproblems which are similar to the original problem. After segmentation, a modified Monte Carlo simulation is used to approximate these rest subproblems, which is called 'in corners' part. Due to the quite small size of these 'in corners' part, when applying modified Monte Carlo simulation, the number of sampling points would be much less than the original method. With the reduction of the number of sampling points, the running time is also reduced. This method is a combination of the exact calculation method and the approximation method, and thus, we call it 'partial precision and partial approximation'.

To better understand the proposed hypervolume approximation method, theoretical analysis about this method is given in this paper. Meanwhile, a series of experiments are conducted to systematically investigate its efficiency. To be specific, we compare the proposed method with three exact calculation methods (QHV, WFG, and HBDA) and an approximation method (FPRAS) on four widely-used test instances. Simulation results show that the proposed algorithm has low running time in high dimensional hypervolume calculation. The proposed algorithm can normally achieve about ten times faster than the FPRAS algorithm which is the most efficient algorithm in more than 10 dimensional space so far. Furthermore, we propose a method to find the solution that contributes the least to hypervolume in a population, which is often called the incremental version. We apply the proposed method into the framework of SMS-EMOA [5] and compare it with NSGA–III [15], MOEA/D [46], HypE [2], IBEA [48] and the incremental version of IWFG and exQHV in the framework of SMS-EMOA. By comparing, the proposed algorithm can achieve an acceptable performance in respect of both runtime and the quality of solutions.

In summary, we have the following contributions and innovations in this paper.

- A fast hypervolume approximation algorithm is proposed that accelerates the hypervolume calculation with a steady accuracy error, through a segmentation strategy and the Monte Carlo simulation.
- Theoretical studies of the proposed hypervolume approximation method and some others are conducted.
- Base on the above fast approximation method, an incremental algorithm is proposed for many objective optimization.

This paper is organized as follows. Section 2 briefly introduces the basic concepts of multiobjective optimization and hypervolume. Section 3 gives a detailed description about the proposed hypervolume approximation method. Theoretical analysis is presented in Section 4. Simulation experiments are conducted in Section 5. Section 6 concludes this paper. Appendix A proposes an incremental hypervolume algorithm of the proposed method, and conducts comparative experiments with other EMO algorithms.

2. Problem definition

Without loss of generality, the multiobjective optimization problems (MOPs) which involve several conflicting objectives to be optimized simultaneously, can be stated as follows:

$$\begin{aligned} &\min \quad F(x) = (f_1(x), f_2(x), \ldots, f_d(x))^T \\ &s.t. \quad x \in \Omega, \end{aligned} \tag{1}$$

where $\Omega \subset R^m$ is the decision space and m is the dimensionality of the decision variable x. F: $\Omega \to R^d$ consists d real-value objective functions and R^d is called the objective space. When $d \geq 4$, the problem (1) is regarded as a many-objective optimization problem.

Let $u = (u_1, \ldots, u_d)^T$ and $v = (v_1, \ldots, v_d) \in R^d$ are images of two solutions in the objective space, u is said to dominate v ($u \prec v$) if and only if $u_i \leq v_i$ for all $i = 1, \ldots, d$ and $u \neq v$. x^* is called Pareto optimal solution if there is no solution $x \in \Omega$ such that $F(x) \prec F(x^*)$. The set of all the Pareto-optimal solutions in Ω is called the Pareto set (PS) and the set of all the Pareto optimal objective vectors is the Pareto front (PF), there is a one-to-one correspondence between PS and PF.

According to the definition above, in the d-dimensional objective space, we can define the dominated region namely hypervolume measure for given any set $\mathcal{F}$ of n vectors as follows

$$HV(\mathcal{F}, B) = VOL\left(\bigcup_{(f_1, f_2, \ldots, f_d) \in \mathcal{F}} [f_1, b_1] \times \ldots \times [f_d, b_d] \right), \tag{2}$$

where $B = (b_1, b_2, \ldots, b_d)$ is a computational bound of $\mathcal{F}$ which is dominated by all the vectors in $\mathcal{F}$ and VOL denotes the Lebesgue measure. Particularly, when $n = 1$ and supposes $\mathcal{F} = \{p\} = \{(f_1, \ldots, f_d)\}$, we have the equation

$$HV(\mathcal{F}, B) = HV(\{p\}, B) = \prod_{i=1}^{d} (b_i - f_i), \ p \in \mathcal{F}. \tag{3}$$

By definition, hypervolume measure is the volume of the union of the boxes (or hypercubes), a box is made up by the region between a vector and the computational boundary. This metric value is to be maximized, and the larger measure of a population, the better quality of population it is. As shown in Fig. 1, the shadow area is the hypervolume measurement of the set (or population) $\mathcal{F} = \{a, b, c, d, e\}$, namely $HV(\mathcal{F}, B)$.

Generally, hypervolume based evolutionary algorithms utilize this equation (Eq. (2)) as the performance indicator for selecting offspring. For instances, given a set $\mathcal{F}$ of n vectors in objective space, the hypervolume contribution (or called exclusive hypervolume) of a vector $p \in \mathcal{F}$ in SMS-EMOA [5], is defined as follows

$$ExcHV(p, \mathcal{F}, B) = HV(\mathcal{F}, B) - HV(\mathcal{F} \backslash p, B). \tag{4}$$

However, as mentioned in Section 1, it is difficult to gain the result of hypervolume contribution just by this equation (Eq. (4)) in the high-dimensional spaces. Therefore, to alleviate the difficulty, some algorithms were proposed. HypE [2] estimates the hypervolume contribution by means of Monte Carlo simulation, FV-MOEA [27] adopts the contribution update strategy rather than recalculating them. Despite all those efforts, they actually push the target of gaining the value of $ExcHV(p, \mathcal{F}, B)$ forward to a certain degree.

3. The proposed fast hypervolume approximation algorithm

In this section, we show how the proposed fast hypervolume approximation calculation method works in high dimensional objective space. The algorithm consists of two parts: the segmentation (precision) part and the approximation part, which can be summarized to calculate the following equation

$$HV(\mathcal{F}, B) = V_p + V_{\mathcal{Q}}, \tag{5}$$

where V_p represents the segmentation part which is introduced in first subsection, and $V_{\mathcal{Q}}$ represents the approximation part which is introduced in second subsection correspondingly.

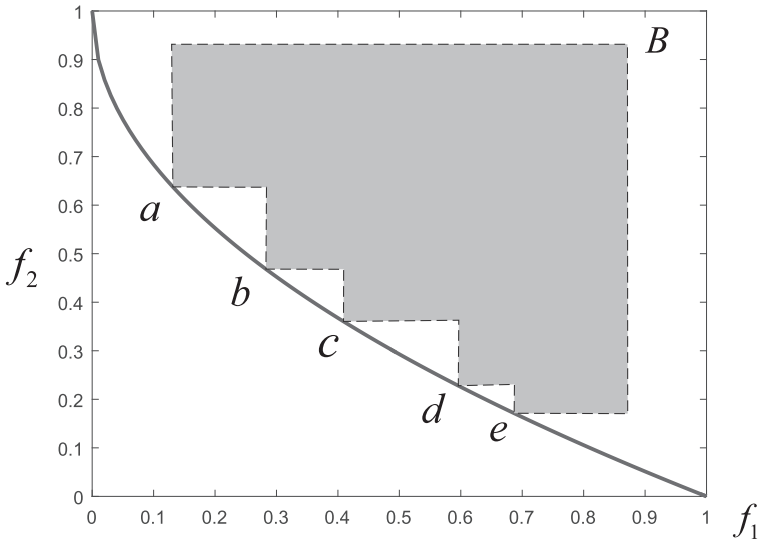


Fig. 1. The shadow area is the value of $HV(\mathcal{F}, B)$, $\mathcal{F} = \{a, b, c, d, e\}$.

This approach can achieve fast calculation because the number of Monte Carlo sampling points decreases in the approximation part, while the segmentation part can improve accuracy. Therefore, we call it 'partial precision and partial approximation (PPPA)' method.

3.1. A Segmentation Strategy and exact calculation algorithm

In this part, we first introduce the segmentation part, which is actually a recursive pivot divide and conquer process. The pivot divide and conquer technique consists of three steps:

1. Select a pivot point which is processed easily and excluded from the recursion;
2. Divide the original problem according to the pivot, classify other points into the possible sub-problem;
3. Recursively solve each of the sub-problems in a 'smaller' regions of space, and add up the hypervolumes.

More details about the above method can be found in QHV algorithm [39], which decomposes the original problem into $2^d - 2$ sub-problems by a segmentation method and loops the above steps. In this paper, the same technique is adopted, but we realize it in another segmentation methodology and optimize its realization details. The novelty of the proposed method is mainly reflected in the operations of splitting and pivot selecting, which includes the pivot point selection, the specific steps of segmentation, and reduction of iteration times. Algorithm 1 (PDCH, Pivot Divide to Calculate Hypervolume) gives the procedure of this segmentation method, where $B = (b_1, b_2, \ldots, b_d)$.

Main inputs of PDCH Algorithm are: a set $\mathcal{F}$ which has n points and the computational boundary B. The output of this algorithm is the hypervolume measurement for $\mathcal{F}$. As shown in Algorithm 1, before the recursion, the procedure would judge whether it steps into the recursion or not, according to the number of points n (Line 2). If the recursive condition is not meet, the recursion step would not be executed, instead, the procedure returns the result of Eq. (3) (Line 3) directly. Otherwise, some preparations of recursion would be done (Lines 5–15). Firstly, it finds out a pivot point $p \in \mathcal{F}$ which satisfies the condition of having the maximum volume (Line 5 in Algorithm 1), simultaneously calculates the volume of p and adds them to the hypervolume measurement (Lines 5–6), it is easy to complete because a series of multiplications are done here just as shown in Eq. (3), with the complexity of $O(d)$. This pivot would be used for comparison and segmentation later. After that, the divide and conquer technique would be looped d times (Lines 8–14) according to different dimensions. In this part, Lines 8–13 is a segmentation strategy, namely the divide step, and Line 14 is the recursive step. The segmentation strategy is the core of PDCH Algorithm, a subset $\mathcal{Q}$ is initialized to record the segmented points (Line 8), then the points in $\mathcal{F}$ which are greater than the pivot in current dimension would be divided into two parts: one part is added to subset $\mathcal{Q}$, while the other part replaces the original points in $\mathcal{F}$ and then joins to next segmentation (Lines 10–12). Finally, the $\mathcal{Q}$ is solved by the recursion (Line 14) and return the recursive result (Line 17).

To make it easy to follow the segmentation strategy, Fig. 2 illustrates how it works in the 3-dimension space for maximizing the objectives. Notice that in Fig. 2, we rotate the set 180 degrees and let $B = (0, \ldots, 0)$, then the problem (1) is turned into a maximizing problem. It is accomplished by segmenting the original geometric problem into a cube (or hypercube) and 3 (or d) subproblems along the direction of each dimension.

It can be easily induct that the segmentation of each dimension produces one subproblem. In d-dimensional objective space, once all the dimensions are segmented as described above, d subsets can be obtained. Compared to QHV which generates $2^d - 2$ subsets, the subsets quantity of the proposed strategy is greatly reduced.

Algorithm 1 Pivot Divide to Calculate Hypervolume in d Dimensional Space.

Input: Population $\mathcal{F}$, A set of n points; B, Computational boundary vector, all points in $\mathcal{F}$ dominate B
Output: $HV(\mathcal{F}, B)$, the hypervolume measurement for $\mathcal{F}$

1: **function** PDCH($\mathcal{F}, B$)
2: **if** $n = 1$ **then**
3: Calculate the Eq. ~(3): $hv^* \leftarrow HV(\mathcal{F}, B)$;
4: **else**
5: Find the pivot $p \in \mathcal{F}$: $p \leftarrow \underset{a \in \mathcal{F}}{\text{argmax}}\ HV(\{a\}, B)$;
6: Calculate the volume of p: $hv^* \leftarrow HV(\{p\}, B)$;
7: **for** $i \leftarrow 1$ **to** d **do**
8: Initialize the subset $\mathcal{Q}$: $\mathcal{Q} \leftarrow \emptyset$;
9: **for all** a in $\mathcal{F}$ **do**
10: **if** $a_i < p_i$ **then**
11: $a_i \leftarrow b_i - (p_i - a_i)$; $\mathcal{Q} \leftarrow \mathcal{Q} \cup a$; $a_i \leftarrow p_i$;
12: **end if**
13: **end for**
14: Recursion: $hv^* \leftarrow hv^* +$ PDCH($\mathcal{Q}, B$);
15: **end for**
16: **end if**
17: **return** hv^*;
18: **end function**

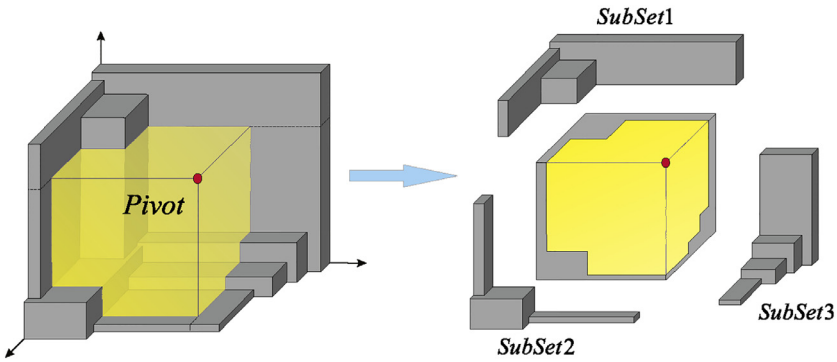


Fig. 2. Illustration of dividing a set in 3D case for maximizing problems.

After d segmentations, the rest of n points are quite easy to process. Fig. 2 shows that these rest points are distributed on the plane which is composed of pivot and coordinate axis, so all of them are dominated by the pivot (Because Fig. 2 is for maximizing the objectives, the point having bigger objective value dominates the point having a small objective value). Since those dominated points do not have any effect on the hypervolume measurement, they could be thrown away without any calculations or processing. Comparing with other representative recursive algorithms, this segmentation method is even not involved non-dominated sorting, which means it would save a fair bit of computational efforts.

Besides, the condition of finding the pivot in line 5 can be changed as required, and we also use the maximum-minimum method shown in Eq. (6) instead. In Eq. (6), each point would be estimated by a fitness which is the maximum value of different dimensions, and we select the point as pivot which has a minimum fitness. It can be guaranteed that the pivot is the midpoint of the set.

$$p \leftarrow \underset{a=(a_1,\ldots,a_d)\in\mathcal{F}}{\text{argmin}}\ max\{a_1, \ldots, a_d\}. \tag{6}$$

Notice that, this segmentation strategy also contributes to approximation of hypervolume measurement. After k layers recursion, d^k subsets can be obtained. Here the concept "layer" means the depth of recursion in segmentation process, which is often used in conjunction with the variable k. Furthermore, it would calculate a new volume of pivot and add to the total measurement of hypervolume in each layer recursion, so just as the Eq. (5) mentioned, we have the formula that

$$HV(\mathcal{F}, B) = \sum_{p\in\mathcal{P}} HV(\{p\}, B) + \sum_{i=1}^{d^k} HV(\mathcal{Q}_i, B) = V_p + V_{\mathcal{Q}}, \tag{7}$$

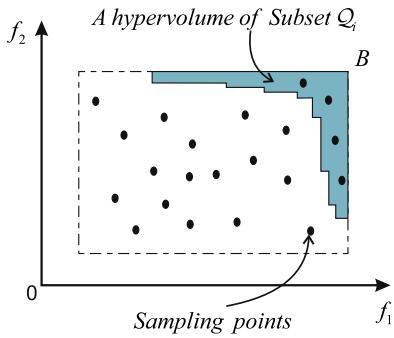


Fig. 3. If the size of a subset $\mathcal{Q}_i$ has a small proportion in sampling range, most of sampling points may tend to beyond the set and become useless for the calculation.

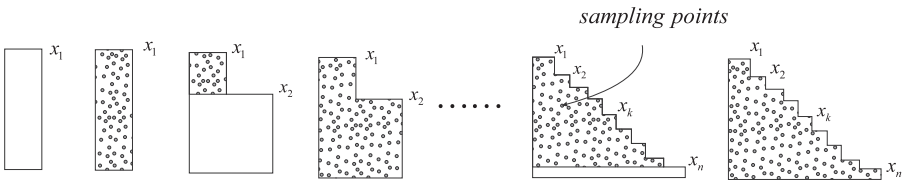


Fig. 4. Illustration of the modified Monte Carlo simulation for maximizing the objectives, which just sample within the hypervolume.

where $\mathcal{P}$ is the set of all the pivots and $V_p = \sum_{p\in\mathcal{P}} HV(\{p\}, B)$ is the sum of the volume of all pivots, $\mathcal{P} \not\subset \mathcal{F}$ but all the elements in $\mathcal{P}$ are dominated by $\mathcal{F}$. $\mathcal{Q}_i$ traverses all subsets which are obtained by the k-layer segmentation and $V_\mathcal{Q} = \sum_{i=1}^{d^k} HV(\mathcal{Q}_i, B)$ is the sum of the hypervolume measure of all subsets.

Obviously, the first part of the equation is easy to calculate precisely by a series multiplications, however, it is still difficult to measure the second part in a reasonable time, and that is why we need approximation. If we can propose an approximation strategy to estimate the second part, this equation could be regarded as an calculation of the hypervolume measurement with an error term. The first part is the precise calculation and the second part is an estimation. Meanwhile, the larger k is, the more accurate results are (but with a larger computation cost). Therefore, the approximation strategy and a modified Monte Carlo simulation will be emphatically discussed in next subsection.

3.2. The proposed hypervolume approximation method in high dimensional space

Monte Carlo simulation is an important technique for tackling the high dimensional problems, and using Monte Carlo simulation to approximate hypervolume has been studied by many researchers [2,7,16]. It often consists of three steps:

1. Produce S_1 sampling points in the sampling range $[0, b_1] \times \ldots \times [0, b_d]$;
2. Count the number of sampling points which are dominated by the hypervolume problem (or fell within the internal hypervolume), and denoted as S_2;
3. Then the approximation of hypervolume measurement equals to $(\frac{S_2}{S_1} \cdot HV(0, B))$.

In this part, we improve the Monte Carlo simulation for a better integration when approximating the error term ($V_\mathcal{Q}$) in Eq. (7). Based on above analysis, if the size of set $\mathcal{F}$ has a small proportion in sampling range $[0, b_1] \times \ldots \times [0, b_d]$, most of the sampling points would tend to beyond the set $\mathcal{F}$, and it may cause the waste of computation effort. The subsets $\mathcal{Q}_i$ obtained by Eq. (7) also conform to this situation, which is shown in Fig. 3. Unfortunately, Monte Carlo simulation has disadvantage in computing the above mentioned subsets. Therefore, to overcome this, we propose modified version which associates the number of sampling points with the size of hypervolume. In general, this modified Monte Carlo method just needs to sample within the hypervolume, so when the proportion of subsets size is reducing in the sampling range, the number of sampling points is decreasing as well.

Lines 5–19 is the main procedure of the modified Monte Carlo simulation in Algorithm 2, and Fig. 4 also illustrates this method in the 2-dimension space for the convenience of understanding. First, randomly take a point x_1 out from the set $\mathcal{F}$, generate $\rho \cdot HV(\{x_1\}, B)$ sampling points uniformly between x_1 and the computational boundary B, merge them to the collection $\mathcal{S}$ (Lines 12–17), here ρ is the number of sampling points per unit volume. The ways to uniformly produce the sampling points are varied, and in this paper we adopt the stochastic generation method (more details can be found at Lines 13–17). Second, take the second point x_2 out from the set $\mathcal{F}$, delete the sampling points which are dominated by x_2 in $\mathcal{S}$ (Lines 7–11), at last merge $\rho \cdot HV(\{x_2\}, B)$ sampling points to the collection $\mathcal{S}$, which are generated between x_2 and the computational boundary B uniformly (Lines 12–17). Repeat this process until all the points in $\mathcal{F}$ are taken out, then $\frac{|\mathcal{S}|}{\rho}$ is an estimate of hypervolume measurement (Line 19).

Algorithm 2 Partial Precision and Partial Approximation of Hypervolume.

Input: $\mathcal{F}$ and B like Algorithm 1; k, the current layer of recursion; ρ, the number of sampling points per unit volume
Output: the approximation of hypervolume for $\mathcal{F}$

1: **function** PPPA($\mathcal{F}, B, \rho, k$)
2: **if** $k \leq Iter$ **then** /**Iter* is a preset parameter of max recursive layers */
3: Execute Algorithm 1, but the line 14 is replaced by: $hv^* \leftarrow hv^* +$ PPPA($\mathcal{Q}, B, \rho, k+1$);
4: **else**
5: Initialize the set of Sample: $\mathcal{S} \leftarrow \emptyset$;
6: **for all** $a = (a_1, \ldots, a_d)$ in $\mathcal{F}$ **do**
7: **for all** b in $\mathcal{S}$ **do** /*Delete points in $\mathcal{S}$ which are dominated by a */
8: **if** $b \prec a$ **then**
9: $\mathcal{S} \leftarrow \mathcal{S} \backslash b$;
10: **end if**
11: **end for**
12: Sampling size of a: $s \leftarrow \rho \cdot HV(\{a\}, B)$;
13: /*Produce s sampling points that uniformly distributed within $[a, b]$ and add to $\mathcal{S}$ */;
14: **for** $i \leftarrow 1$ **to** s **do**
15: $c \leftarrow (c_1, \ldots, c_d)$, $c_j = random(a_j, b_j)$, $j = 1, \ldots, d$;
16: $\mathcal{S} \leftarrow \mathcal{S} \cup c$;
17: **end for**
18: **end for**
19: **return** $\frac{|\mathcal{S}|}{\rho}$;
20: **end if**
21: **end function**

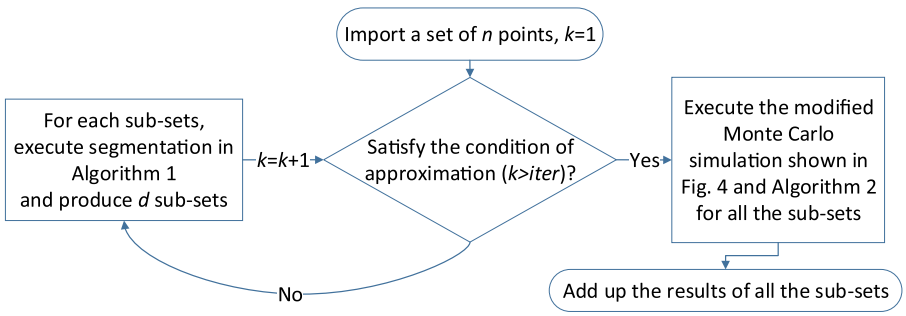


Fig. 5. Flow diagram of PPPA algorithm by integrating segmentation method and Monte Carlo simulation.

Notice that ρ is generally a large positive integer. The determination of ρ depends on a polynomial function about a required error ε, the greater the ρ, the more accurate the measured value would be, and vice versa. More information about ρ can be found in [2,7,16]. The sampling size s of a point $x_i, i = 1, \ldots, n$ must follow the equation which is shown in Line 12, to make sure the validity of result.

In fact, the principle of this method is similar to the relationship between density and volume. With density unchanged, we keep adding (or 'pouring') points into the dominated region which is consisted by the set $\mathcal{Q}_i$ and the boundary B, until the dominated region is saturated. Then the hypervolume measure equals the numbers of total points in the dominated region divided by the density. It can be concluded that when the density (ρ) keeps constant, the algorithm would just sample in the internal hypervolume, and thus it needs less number of sampling points by comparing the original Monte Carlo simulation which must sample in whole sampling range (the whole computational boundary). Therefore, the computational efficiency of this method can increase to some degree.

Finally, by integrating Algorithm 1 and the modified Monte Carlo simulation method, we propose a fast hypervolume approximation method, PPPA algorithm (Partial Precision and Partial Approximation). It is described in Algorithm 2, where *Iter* is the parameter of max recursive layers (often greater than 2), the input k is set to 1 by default and $|\mathcal{S}|$ represents the number of $\mathcal{S}$. For understanding the process of integration, a flowchart is also given in Fig. 5. The complexity of Algorithm 2 is going to be discussed in the following section.

In summary, when combining with the segmentation part and the approximation part, the proposed algorithm has following characteristics and advantages.

- The accuracy of the hypervolume indicator approximation is improved by partially estimating the computational area instead of approximately estimating the whole.

- The runtime of approximation is also improved. Benefit from the small approximation region, the number of sampling points is accordingly reduced, and thus approximation is speeded up.

4. Theoretical analysis of the proposed algorithm

In this section, we theoretically analyze the complexity of the proposed method, and then compare it with the classical Monte Carlo simulation.

4.1. The complexity of algorithm 2

Firstly, we prove that the complexity of Algorithm 2 is $O(dn\rho(V_Q)^{1+\varepsilon} + (dn)^k)$, where the definition of V_Q is the same as the last term (error term) in Eq. (7), ε is an arbitrarily small number and ρ is the number of sampling points per unit volume, and ρ is a large positive integer which has been mentioned above.

Proposition 1. *As described, the returned value hv^* in Algorithm 2 converges to the hypervolume measure of set $\mathcal{F}$. Meanwhile, the computational complexity of Algorithm 2 is $O(dn\rho(V_Q)^{1+\varepsilon} + (dn)^k)$, $V_Q < 1$.*

Proof. The proposition is equivalent to prove that, for any $\varepsilon > 0$ there is an integer M such that

$$|hv^* - HV(\mathcal{F}, B)| < \varepsilon, \quad \text{whenever } \rho > \mathrm{M}.$$

In general, let the sampling boundary be $B = (b_1, \ldots, b_d) = (1, \ldots, 1)$ because arbitrary point set can be mapped in $[0, 1]^d$ by scaling. We first consider the situation of two points in the set, let $\mathcal{F} = \{\mathcal{F}_1, \mathcal{F}_2\}$, $V_i = HV(\{\mathcal{F}_i\}, B)$, $V_i < 1$, $i = 1, 2$, and

$$\frac{S_1}{V_1} = \frac{S_2}{V_2} = \rho,$$

which is done in Algorithm 2 line 12. Combining with Eq. (2), we have the equation $HV(\mathcal{F}, B) = V_1 + V_2 - V_\cap$, where $V_\cap$ is the volume of intersection $\mathcal{F}_\cap = \{\mathcal{F}_1 \cap \mathcal{F}_2\}$.

In order to calculate $V_\cap$, we uniformly generate S_1 sampling points in the sampling range $[\mathcal{F}_1, b]^d$. If let the event A ={the sampling points are dominated by both $\mathcal{F}_1$ and $\mathcal{F}_2$}, then the probability of A equals $P(A) = V_\cap/V_1$, and f_A is the number of occurrences of the event A in S_1 times.

According to law of large numbers, for every $\varepsilon > 0$ there is an integer M such that

$$\begin{aligned} &\left|\tfrac{f_A}{S_1} - P(A)\right| = \left|\tfrac{f_A}{S_1} - \tfrac{V_\cap}{V_1}\right| < \varepsilon, \\ &\text{whenever } \mathrm{S}_1 = \rho V_1 > M. \end{aligned} \tag{8}$$

Because of the sampling boundary $[\mathcal{F}_1, b]^d$, the sampling points are always dominated by $\mathcal{F}_1$, hence we just need to estimate the quantity of sampling points which are dominated by $\mathcal{F}_2$ when we calculate the f_A. This operation is done in Algorithm 2 lines 7–11, and then according to Algorithm 2, we have the following deduction

$$\begin{aligned} hv^* &= \frac{|\mathcal{S}|}{\rho} = \frac{S_1 - f_A + S_2}{\rho}, \\ |hv^* - HV(\mathcal{F}, B)| &= \left|\frac{S_1 - f_A + S_2}{\rho} - (V_1 + V_2 - V_\cap)\right| \\ &= \left|V_\cap - \frac{f_A}{\rho}\right| = V_1\left|\frac{V_\cap}{V_1} - \frac{f_A}{S_1}\right| < V_1\varepsilon < \varepsilon. \end{aligned} \tag{9}$$

Using the mathematical induction, Eq. (9) is easily generalized to the situation of n points in set, If we guarantee

$$\frac{S_1}{V_1} = \frac{S_2}{V_2} = \cdots = \frac{S_n}{V_n} = \rho.$$

$HV(\mathcal{F}, B) = HV(\mathcal{F}\backslash\mathcal{F}_n, B) + V_n - V_\cap$ can also be obtained by inclusion exclusion (IEX), so all the recursive conditions above are established and the proof is completed.

We uniformly generate $S_i (i = 1, 2, \ldots, n)$ sampling points in the sampling range $[\mathcal{F}_i, b]^d$ and add them into the set $\mathcal{S}$, so we just need to compare the dominated relationship between $\mathcal{F}_i$ and the sampling points belong to $\mathcal{S}$. If $\mathcal{S}$ contains $|\mathcal{S}|$ sampling points, the computational complexity is $O(|\mathcal{S}|d)$. Since $|\mathcal{S}|/\rho$ converges to the real value of hypervolume measurement hv, we can get $||\mathcal{S}|/\rho - hv| < \varepsilon$. Thus, $|\mathcal{S}| < \rho(hv)^{1+\varepsilon}$ can be deduced. Such an operation has to be repeated n times, so the complexity of approximation in Algorithm 2 is $O(dn\rho(hv)^{1+\varepsilon})$.

According to Algorithm 1, the cost of each segmentation is $O(dn)$. A segmentation can generate d subsets. After k layers segmentation, d^k subsets are generated, and thus the cost of segmentation is $1 + d + \cdots + d^{k-1} = d^k/(d-1) = O((dn)^k)$.

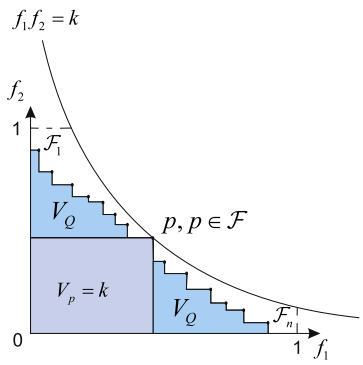


Fig. 6. The contour of the point p and the region of V_p and V_Q, $p \in \mathcal{F}$.

Suppose that the hypervolume measure of subset Q_i is hv_i after segmentation, then the total cost of calculating a subset Q_i is $O(dn\rho(hv_i)^{1+\varepsilon})$. Meanwhile, the cost of calculate all subsets $Q_1, Q_2, \ldots, Q_{d^k}$ is

$$\begin{aligned}\sum_{i=1}^{d^k} O(dn\rho(hv_i)^{1+\varepsilon}) &< O\left(dn\rho\left(\sum_{i=1}^{d^k}(hv_i)^{1+\varepsilon}\right)\right)\\ &< O(dn\rho(V_Q)^{1+\varepsilon}).\end{aligned} \tag{10}$$

Therefore, the complexity of Algorithm 2 is $O(dn\rho(V_Q)^{1+\varepsilon} + (dn)^k)$. In general, $V_Q < 1$. □

4.2. Algorithm 1 segmentation performance

Secondly, we analyze the relationship about the dimension increment and the segmentation performance in Algorithm 1.

In general, we suppose that the segmentation performance of Algorithm 1 mainly manifests on the value of V_Q (the last item in Eq. (7)). According to the Proposition 1, we know that the complexity of Algorithm 2 is related to V_Q. The smaller V_Q is, the faster calculation of Algorithm 2 is, and thus we consider the better segmentation performance of Algorithm 1 is (Algorithm 1 can affect V_Q).

We prove that the segmentation performance of Algorithm 1 is effective because it can reduce the value of V_Q effectively. And thus we deduce that the proposed algorithm outperforms the classical Monte Carlo simulation in theory.

However, It is also shown that a worst case is existed which leads to poor segmentation performance due to the dimension increasing.

Proposition 2. *Assume that the point having the largest volume in the set $\mathcal{F}$ is regarded as the pivot. After the first layer segmentation, the last item V_Q in the Eq. (7) is lesser than e^{-1} in the 2-dimension. It means that the part of hypervolume which we need to approximate by Monte Carlo sampling is lesser than e^{-1}.*

Proof. For readability, we transform the point set appropriately, let $\mathcal{F}_i = B - \mathcal{F}_i$ $(i = 1, \ldots, n)$ and then ignore the influence of the computational boundary. Suppose p is the point having the largest volume in the set $\mathcal{F}$, $p = (p_1, p_2)$, we define a constant k is

$$k = HV(\{p\}) = p_1 p_2,\ 0 < k < 1,\ p \in \mathcal{F},$$

where $HV(\mathcal{F})$ is the same as $HV(\mathcal{F}, B)$. Consider the contour of the hypervolume measurement of the point p, namely $f_1 f_2 = k$. We draw it in Fig. 6 which is a convex curve. The shadow in Fig. 6 is the dominated region (or covered space) of the $\mathcal{F}$ and the union volume of shadow is the hypervolume measure of $\mathcal{F}$. We now consider the following function

$$\begin{aligned} g(x) &= \begin{cases} \frac{k}{x}, & k < x \le 1 \\ 1, & x \le k \end{cases},\\ \varphi_1(k) &= \int_0^1 g(x)\,dx = k + \int_k^1 \frac{k}{x}dx = k - k\ln k. \end{aligned} \tag{11}$$

Because the points of $\mathcal{F}$ always falls between the curve $f_1 f_2 = k$ and the coordinate axis, we can deduce that

$$\begin{aligned} HV(\mathcal{F}) = V_p + V_Q &< \varphi_1(k) = k - k\ln k\\ k + V_Q &< k - k\ln k\\ V_Q &< -k\ln k. \end{aligned} \tag{12}$$

The maximum of above can be obtained by derivation. It is clearly that, when $k = e^{-1}$, $V_{\mathcal{Q}}$ reaches the maximum e^{-1}, so

$$V_{\mathcal{Q}} < e^{-1},$$

and the proof is completed. □

Unfortunately, with the dimension increasing, the segmentation performance of Algorithm 1 decreases.

Similarly, the analysis above can be generalized to the high dimensions, we can found a d-pile integral

$$\begin{aligned} V_{\mathcal{Q}} &< \varphi_d(k) - k \\ &= \varphi_{d-1}(k) + \int \cdots \int_{\triangle} \frac{k}{f_1 f_2 \cdots f_d} d\triangle - k \\ &= k \sum_{i=1}^{d-1} \frac{1}{i!} (-\ln k)^i, \\ \triangle &: \left\{ f_1, \ldots, f_{d-1} = k,\ f_1, \ldots, f_{d-1} = 1,\ f_d = 0,\ \frac{k}{f_1 f_2 \cdots f_d} = 1 \right\}. \end{aligned} \tag{13}$$

It is not difficult to know that the right side of inequality (13) converges to the function $(1 - k)$, and the maximum value about of $V_{\mathcal{Q}}$ tends to 1. With the increase of dimension, the worst-case segmentation performance is gradually decreased in Algorithm 1. Therefore, the more segmentation layers in high dimension are necessary to ensure the calculation precision. In truth, under the condition of given precision, the least segmentation layers is a function about d-dimension, but it is difficult to analyse this relationship and it is our follow-up research work.

Finally, if we put the result into proposition 1, substitute the $V_{\mathcal{Q}}$ with e^{-1} and ignore the second item (much smaller than the first item), we can obtain a worst case upper bound runtime of Algorithm 2 in 2D, which is $O(dn\rho(e^{-1}))$. According to [2,16], the complexity of Monte Carlo sampling simulation is $O(dn\rho)$. Comparing with these two algorithms, we can know that the proposed algorithm is at least e times faster than Monte Carlo simulation in 2D.

5. Experiment on calculation of hypervolume indicator

In order to demonstrate the feasibility and effectiveness of the proposed method, we do a series of comparative experiments by comparing the performance of the proposed method[1] with four previous hypervolume calculation method: QHV, WFG, HBDA, and FPRAS.

5.1. Introductions of comparison algorithms

The detailed introductions of the comparison algorithms are as follows.

QHV [39,40] (Quick hypervolume) is currently the fastest algorithm when calculating a single union area of n points, it uses the divide and conquer strategy like QuickSort, which splits the objective space into $2^d - 2$ hypercuboids corresponding to each possible combination of comparisons on each objective, where a coordinate may be $>$ or $\geq$ than the corresponding coordinate of the pivot point s', with the exception of two combinations corresponding to the areas dominated and dominating s'. Although the worst case of this method is $dn\Theta(2\text{nd})$, when solutions are uniformly distributed on a hyper-sphere or hyper-plane, the time complexity of QHV is $O(dn^{1.1}\log^{d-2} n)$. Recently, an incremental version of QHV (exQHV) was extended to compute the contributor of each point and speed QHV up by the parallel computation method [40], which means that the QHV algorithm also can be integrated into the hypervolume based evolutionary algorithms efficiently.

WFG [43,44] (Walking Fish Group) combines the point-wise processing used in LebMeasure [19] with a new technique for calculating the exclusive hypervolume of a point. The exclusive hypervolume of a point p relative to a set S is the difference between the inclusive hypervolume of p and the hypervolume of S after each point has been limited to be no better than p in any objective. Even though some studies et al. [43,44] proved that the worst-case time complexity of WFG is $O(2^{n-1})$, the practice of WFG is much faster than the theory because most of other solutions probably have zero contribution for this solution and hence would not participate into the hypervolume calculation. WFG has been generally accepted as the fastest exact hypervolume calculation method [37,43,44]. In [14], the Incremental algorithm (IWFG) has proposed for determining which point in a set contributes least to the hypervolume without calculating all the exclusive hypervolume of points in the set S.

HBDA [28] (Hypervolume Box Decomposition Algorithm) is a new approach to the computation based on partitioning the dominated region into a set of axis-parallel hyperrectangles or boxes, the time complexity of the nonincremental algorithm is $O(n^{\lfloor \frac{d-1}{2} \rfloor + 1})$, but the calculation efficiency is reported faster than the above algorithms in some instances. Although the complexity of algorithms above is still exponential with the number of dimension, their runtime is acceptable when $d \leq 10$.

[1] The C++ Mex source code of the proposed PPPA is available online: https://github.com/tang576225574/hypervolume or http://www.lhl-gdut.cn/lhl/.

FPRAS [7] (Fully Polynomial-time Randomized Approximation Scheme) is an algorithm to gain an approximation value of hypervolume by means of statistics method. Its complexity is $O(dn/\epsilon)$ with the error of $\pm\sqrt{\epsilon}$. Although the result may miss regions with 25% probability, this probability can be further reduced.

Because PPPA and FPRAS are both hypervolume approximation methods, we mainly consider FPRAS algorithm in our experiments.

5.2. Testing instances and parameter settings

All algorithms are tested on stable sets of points generated according to several schemes, we consider the following four instance types:

C) Concave or so-called spherical instances;
X) Convex instances;
L) Linear instances;
D) Degenerated instances;

Instances are obtained by drawing uniformly points from the open hypercube $(0, 1)^d$. Then each point z is modified as follows:

C) $z_j \leftarrow \frac{z_j}{\sqrt{\sum_{k=1}^{d} z_k^2}}$, for each $j \in \{1, \ldots, d\}$, i.e. the component values of z are divided by their ℓ_2–norm.
X) $z_j \leftarrow 1 - \frac{z_j}{\sqrt{\sum_{k=1}^{d} z_k^2}}$, for each $j \in \{1, \ldots, d\}$.
L) $z_j \leftarrow \frac{z_j}{\sum_{k=1}^{d} z_k}$, for each $j \in \{1, \ldots, d\}$, i.e. the component values of z are divided by their ℓ_1–norm.
D) $z_j \leftarrow z_l$, for each $j \in \{l+1, \ldots, d\}$, then $z_j \leftarrow \frac{z_j}{\sqrt{\sum_{k=1}^{l} z_k^2}}$, for each $j \in \{1, \ldots, d\}$, l is the degree of degradation, in experiment we let $l = \lceil \frac{d}{2} \rceil$.

In particular, for type (C) and (X) instances, we follow the suggestion of Russo and Francisco in [39] to project uniformly distributed points on a hypersphere, instead of using the instances of Deb in [17] (DTLZ). The degenerated (D) instances are like the degenerated dataset appeared in [39], but they have a slight different in the distribution of points. The reference point we choose is the all-ones vector in the minimization cases and the null vector in the maximization. To cope with the restrictions on the optimization direction, we generate instances primarily for the minimization case. Meanwhile, for the maximization cases, we consider a symmetric instance by taking for each point z and component j the complement $1 - z_j$.

We generate all the instances for each $d \in \{5, 8, 10, 13, 15\}$ and $n \in \{100, 115, 130, \ldots, 500\}$. The implementations are run up to 20 times on small instances to obtain significant computation times. The reported results are thus averaged. Following the suggestion of Bader and Zitzler [2], 10, 000, 000 sampling points per unit volume are used in the proposed algorithm. The max segmentation recursive layer *Iter* is set to 6 in the proposed algorithm. The following formula is given to compute the accuracy

$$Accuracy = \frac{|hv^* - HV(\mathcal{F}, B)|}{HV(\mathcal{F}, B)},$$

where $HV(\mathcal{F}, B)$ is the exact hypervolume measure of the set $\mathcal{F}$ and hv^* is the approximation by proposed Algorithm 2 or Monte Carlo simulation (FPRAS).

We implement the Algorithm 2 in C++, and the implementations provided by the authors are used for the above algorithms [7,28,39,44]. All implementations are compiled using GCC 4.9.0 and performed under twenty Linux CentOS 7.0 with Intel Xeon CPU E5–2640 v4 at 2.40 GHz without parallel computation and with 80GB of RAM.

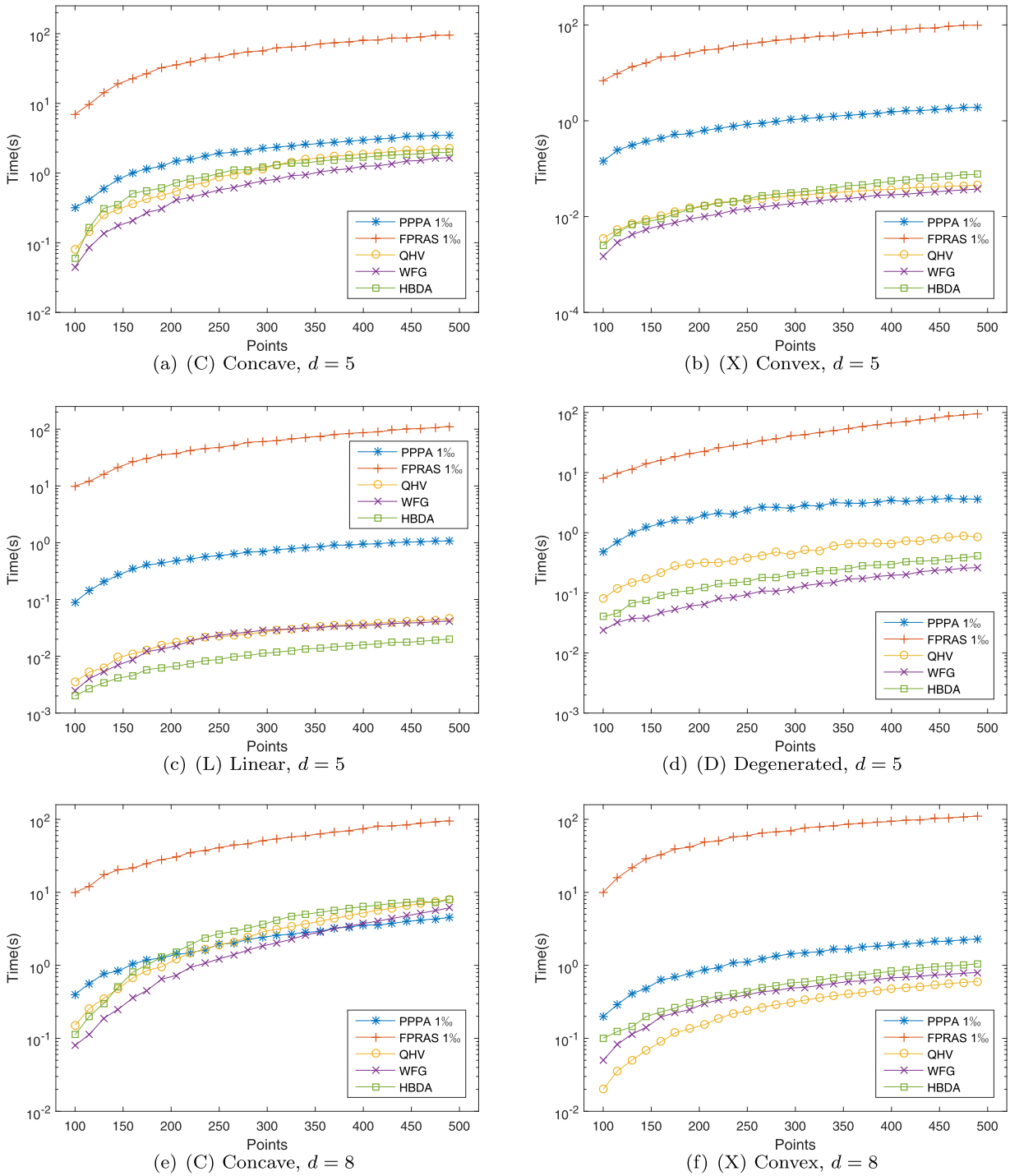
5.3. Results and discussion about runtime

Figs. 7–10 show the running time comparison for the four kinds of different non-dominated sets, and Fig. 11 shows the accuracy comparison.

As for the trends of dimensionality, we can see that the running time of QHV, WFG and HBDA algorithm has a exponentially increasing trend on all instances and all points size. While the proposed algorithm and the FPRAS do not change significantly, which is in accord with the analysis above.

When $d \leq 8$, WFG and QHV algorithm can complete calculation within a very short time, which is often less than 1 s, some instances such as convex and degenerated instances can reach less than 0.1 s (see the Figs. 7 and 8). Compare with the exact calculation methods, our algorithm does not show the superiority. However, the gap is not too obvious, the proposed algorithm completes calculation for about 1 s, even though the number of testing points are more than 400, the running time is remaining to be less than 5 s, which is acceptable too.

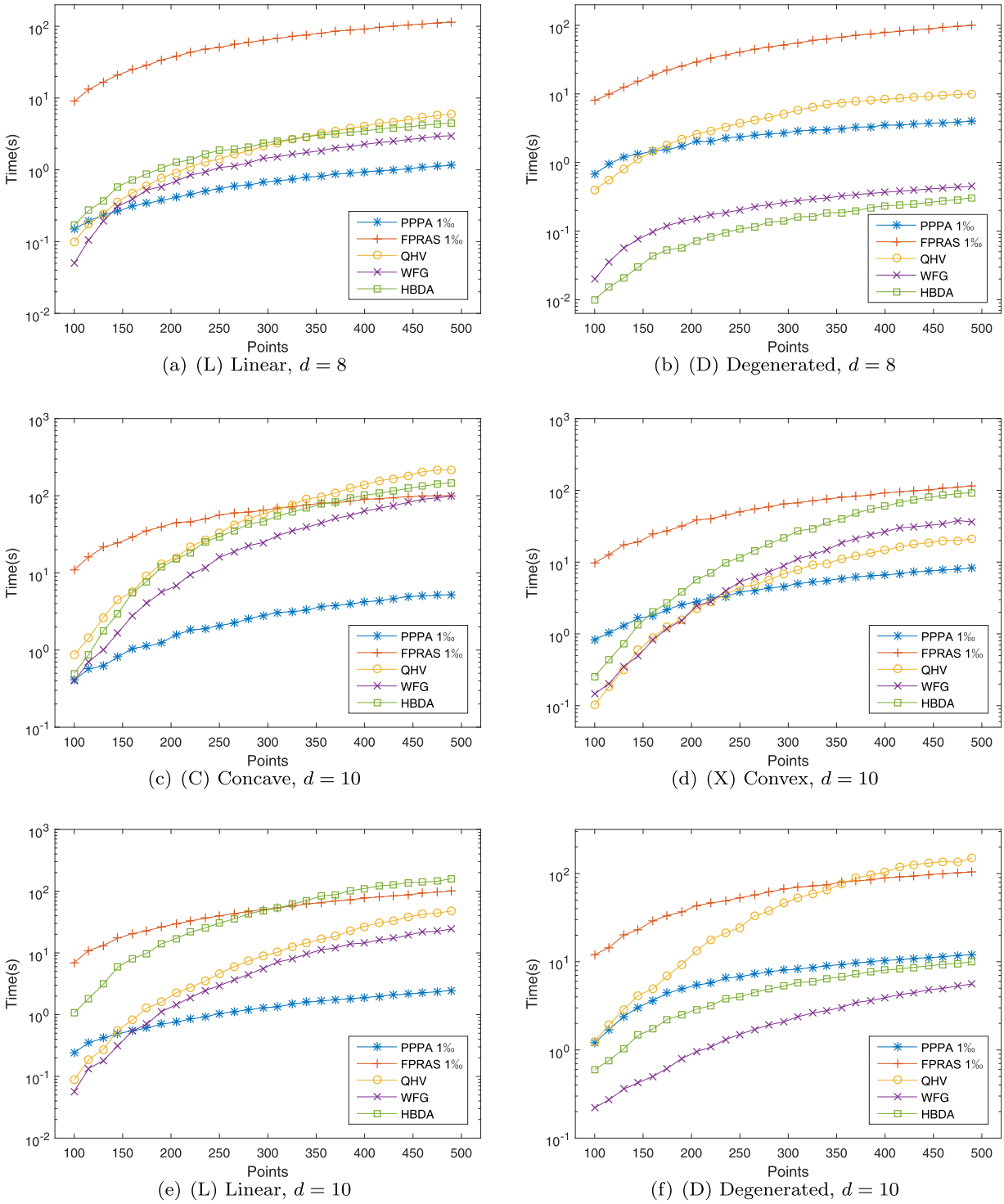
When $d = 10$, the advantage of PPPA algorithm is becoming obvious, and the three algorithms begin to have a long running time. The runtime of them has reached more than 10 s, especially when the testing points exceed 250. Only the degenerated instances can kept at a low runtime (From 0.3 to about 7 s by WFG algorithm according to Fig. 8).



(a) (C) Concave, $d = 5$
(b) (X) Convex, $d = 5$
(c) (L) Linear, $d = 5$
(d) (D) Degenerated, $d = 5$
(e) (C) Concave, $d = 8$
(f) (X) Convex, $d = 8$

Fig. 7. The comparison of running time for 5D and 8D on different instance sets (s).

When $d = 13$, because of the exponential complexity runtime, QHV and WFG algorithms are basically useless. They cost almost 10,000 s for concave instances at 500 testing points, and the other instances are about 1000 s (more information can be referred to in Fig. 9). When $d = 15$, the exact calculation algorithm has not been able to test the running time for all the four instances (more than 10,000 s). So only the FPRAS algorithm is compared with the PPPA algorithm in the case of 15 dimensional space.



(a) (L) Linear, $d = 8$

(b) (D) Degenerated, $d = 8$

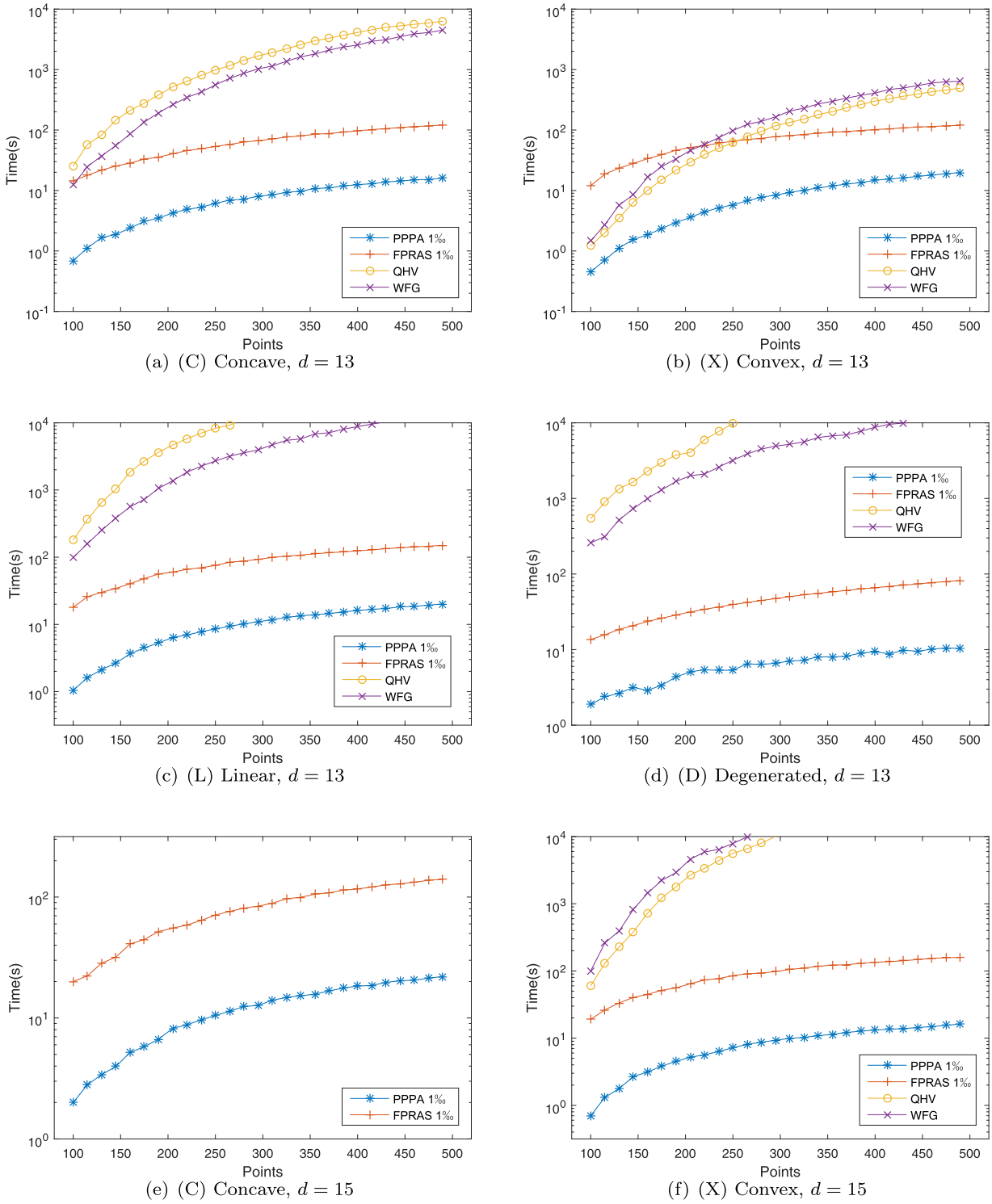
(c) (C) Concave, $d = 10$

(d) (X) Convex, $d = 10$

(e) (L) Linear, $d = 10$

(f) (D) Degenerated, $d = 10$

Fig. 8. The comparison of running time for 8D and 10D on different instance sets (s).



(a) (C) Concave, $d = 13$

(b) (X) Convex, $d = 13$

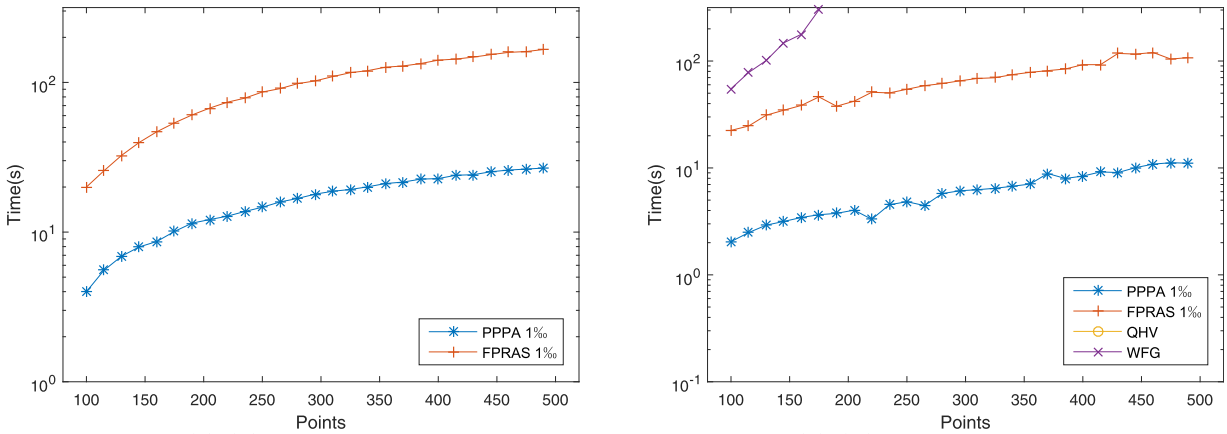
(c) (L) Linear, $d = 13$

(d) (D) Degenerated, $d = 13$

(e) (C) Concave, $d = 15$

(f) (X) Convex, $d = 15$

Fig. 9. The comparison of running time for 13D and 15D on different instance sets (s).



(a) (L) Linear, $d = 15$ (b) (D) Degenerated, $d = 15$

Fig. 10. The comparison of running time for 15D on different instance sets (s).

On the other hand, when $d \geq 10$, the runtime of PPPA algorithm keeps less than 20 s even in 500 testing points. Generally, when the number of testing points is less than 200, PPPA can complete calculation within a few seconds (about 5 s), and is maintaining a steady state which has tiny fluctuation curves for all the four instances. Furthermore, as it is shown in Figs. 9 and 10, after $d \geq 13$ our algorithm runs far shorter than other algorithms, and becomes the fastest algorithm for all testing.

Similarly, as an approximation algorithm, FPRAS also becomes competitive in runtime when $d > 10$. This algorithm is extremely sensitive to the error ε, changing it to 0.1 yields a more than 100 times speed-up in practice, however, it is meaningless to do so because the calculation result is suspect. Therefore, we choose a fixed reasonable ε to show how the performance evolves, the figures show the FPRAS often still needs more than 100 s. With the same condition of precision level, our algorithm is ten times or more faster than it. So we can educe that the proposed algorithm has distinct advantage in speed comparison, the PPPA algorithm omits most geometrical area which originally needs to spend lots of computing resources for approximation, but in fact it is unnecessary to do that. To sum up, the general trends of dimensionality show the proposed algorithm is insensitive to the dimensional increasing (the change is small and mild), and this strategy works.

As for different type of instances, it can be found that the performances of exact calculation algorithms (QHV, WFG and HBDA) show a huge difference in dealing with different instances. It is determined by their division strategy. QHV has a better effect on convex instances (refer to Fig. 7(f) and Fig. 8(d)), but is not good for degenerated instances. HBDA is poor in the performance of linear instances but faster than QHV for degenerated instances (reference Fig. 8(b) and (f)). WFG is dominant in 3 instances. From the overall effect, WFG is better than the other two algorithms, QHV algorithm is the second best, while HBDA algorithm is the last.

Although the performance of WFG on these four instances is promising [43,44], instance can be designed to makes the performance of WFG worst [28,44]. These instances is not considered in our experiment because we mostly focus on the instances with representative PF shapes rather than special instances. The shape distribution of testing cases in our experiment is the most frequently used in the field of EMO. On the other hand, it can be seen that PPPA and FPRAS are not very sensitive with the shape of test instance. The runtime of these two algorithms basically does not fluctuate whatever the distribution of testing set is. It is in line with the above analysis. Therefore, the hypervolume approximation algorithms are steady-state algorithms which can adapt to all kinds of instances, but the exact algorithms are not.

Lastly, the trends are discussed in term of the number of testing points. From Figs. 7 and 8, the increasing of three accurate calculation algorithms (QHV, WFG and HBDA) are drastic, which generally shows a trend of polynomial change (It can be observed from the slope of the curve). And no matter which instance is, the runtime of them would increase more than 100 times when the number of testing points increase from 100 to 500. This span may be even higher in high dimensions ($d \geq 13$).

By contrast, the trends of FPRAS are a linear increasing for each graph. Similarly, PPPA would also increase linearly in theory (time complexity). More exactly, the increase of PPPA would generally be a bit larger than that in linear increments, which we think is influenced by the segmentation part. But from the figures we do not observe this difference, and the curve trends are basically the same as that of the FPRAS. We think that this may be a small incremental amplitude that is not shown in the experiment, or it can be ignored because of the small numbers of testing points.

We have some comments about some disadvantages of the proposed algorithm. The proposed algorithm has a fluctuate runtime. This fluctuation is under control, but it is unpredictable because it may be influenced by the distribution of instances. On the other hand, FPRAS does not have this problem. Another problem is about the parameter setting of max segmentation recursive layer k. If k is too small, the accuracy is not good enough, otherwise, the speed of calculation de-

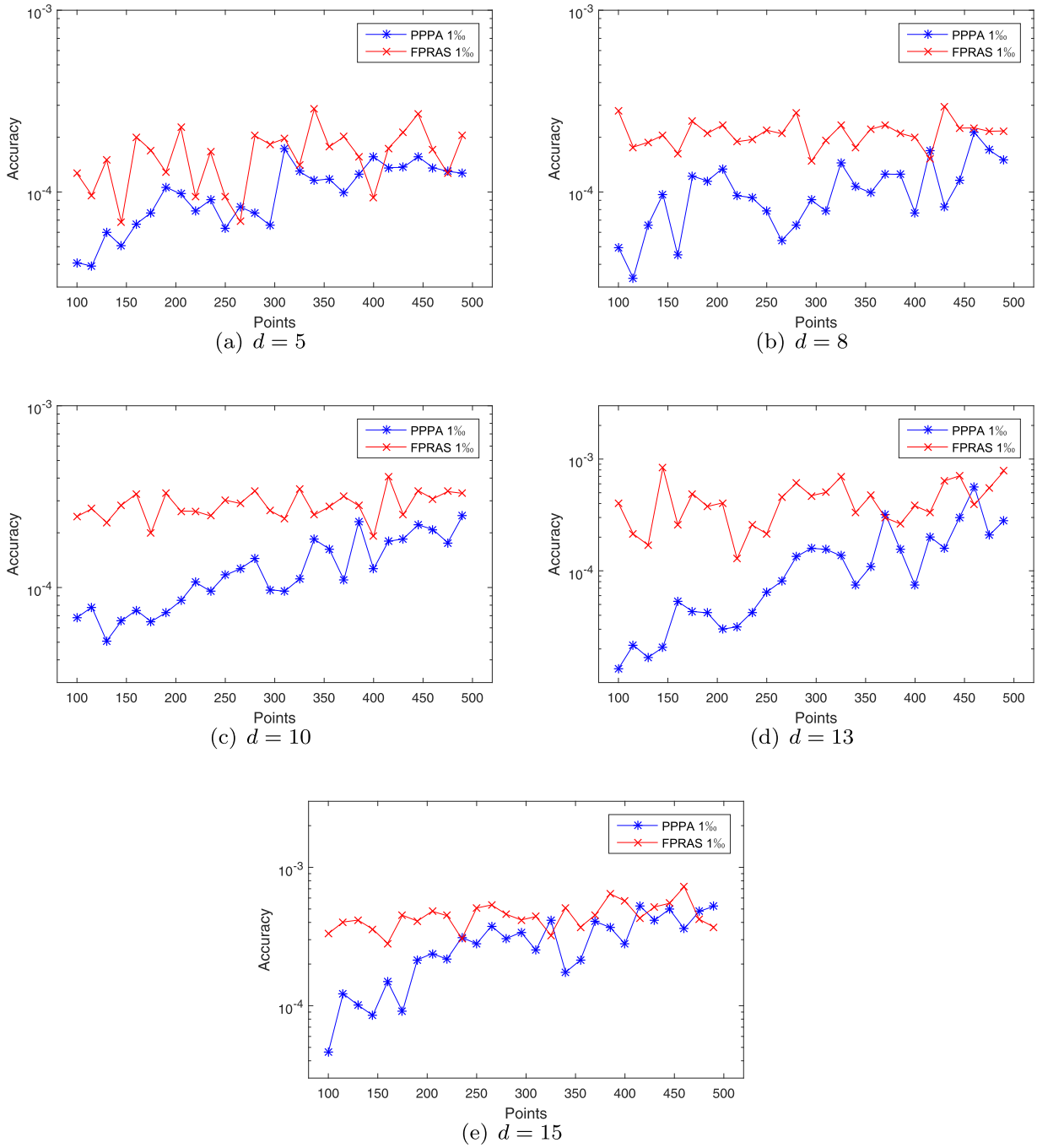


Fig. 11. The comparison of relative error between PPPA and FPRAS on different dimensions.

creases. It is a best possible trade-off problem, and thus, we suggest that k is set about 5 through a number of experiments when $d \leq 15$. We will analyze this quantitative methods theoretically in later studies.

Despite those minor problems, the proposed approximation algorithm is faster than other popular algorithms. We therefore believe that it is a advisable scheme so far and a significant contribution on the hypervolume calculation problem.

5.4. Discussion about errors

The proposed algorithm is not only fast in calculation, but also has stable and high accuracy. Fig. 11 shows the accuracy comparison between PPPA and FPRAS based on the dimensions. Table 1 shows part of the median data in Fig. 11. Because

Table 1
The median of relative errors obtained by PPPA and FPRAS on different dimensions (‰).

No. of	5D		8D		10D		13D		15D	
Points	PPPA	FPRAS	PPPA	FPRAS	PPPA	FPRAS	PPPA	FPRAS	PPPA	FPRAS
100	0.040881	0.126365	0.049608	0.278006	0.068280	0.246137	0.013302	0.402787	0.046137	0.331841
145	0.050392	0.067870	0.097065	0.205291	0.065495	0.283836	0.020662	0.837325	0.085209	0.357239
190	0.106255	0.128726	0.115016	0.210203	0.072771	0.329355	0.041995	0.376636	0.214341	0.409151
205	0.097454	0.226066	0.134364	0.232979	0.084506	0.263620	0.029959	0.404661	0.238294	0.481243
250	0.063129	0.093778	0.078258	0.218728	0.116861	0.302370	0.064531	0.214537	0.280037	0.508030
295	0.065597	0.183275	0.090238	0.147586	0.097341	0.265523	0.159046	0.470122	0.339449	0.416712
340	0.115601	0.286216	0.107431	0.176598	0.184073	0.250766	0.075353	0.333470	0.174518	0.510956
400	0.156245	0.093154	0.077020	0.199727	0.126630	0.191028	0.074450	0.381924	0.279892	0.570658
445	0.156441	0.269398	0.115926	0.224622	0.221406	0.341402	0.300171	0.707582	0.498379	0.549770
490	0.126961	0.204469	0.149809	0.216748	0.247974	0.329774	0.281810	0.780296	0.523038	0.370038

the differences in the computational errors of the 4 instances are very small, the data are not shown individually. Instead, we show the average values of them. In addition, QHV, WFG and HBDA are the exact calculation methods which can gain the accurate results without random errors. Thus, the comparisons are just conducted between the approximated PPPA and FPRAS methods, and the rest three algorithms are ignored. Obviously, the existence of random errors is one of disadvantages of approximation methods, but it is common in high dimensional calculations.

Fig. 11 shows that both of these two algorithms can almost meet the requirement of the error range which is set as 1‰(The relative errors of both two algorithms are less than 1‰). It was reported by Bringmann [7] that the results of FPRAS would out of the error range with only 25% probability in theory, and we do not have similar proof, so in these case we believe the FPRAS is more credible. However, Fig. 11 shows the errors of proposed algorithm are usually smaller than FPRAS, which suggests that PPPA algorithm is more accurate and stable than FPRAS.

When $d \leq 10$, Fig. 11 shows that the fluctuations of the two algorithms are in great randomness. Because the sampling points in Monte Carlo simulation are randomly generated, it may causes this fluctuation when the relative errors reach to some degrees (often less than 10^{-4}). Moreover, it can be deduced that as long as the Monte Carlo approximation algorithm is used (the only approximation method of this problem currently), there always would be a similar fluctuation in the accuracy, and no exception for these two algorithms. This randomness is acceptable and can be ignored because the discussion above is under the condition of a required accuracy. On the other hand, when $d \geq 10$, although the precision curve also shows large random volatility, the overall curve presents an upward trend. That is a noteworthy thing that the accuracy of PPPA algorithm has a declining trend when the number of testing points increasing, but rarely occurs on FPRAS. Although the loss tendency is gentle, it reflects the defect to some degree. The reason for this upward trend may be related to the reduction of the number of sampling points in the algorithm. Although theoretical analysis shows that this reduction would not affect the accuracy of the calculation, experiments show that this method would still reduce the accuracy to some extent. However, it can also be seen from the experiment results that the increase of calculation error is not continued, but disappearing gradually. When the relative error of PPPA calculation exceeds 10^{-4}, the effect of this error expansion has gradually disappeared.

5.5. Statistical analysis

There are some statistical analyses conducted among these algorithms in terms of error rates and runtime. We test two hypothesis testing and thus proving the significant differences between PPPA algorithm and other compared algorithms. These two methods are Wilcoxon rank sum test and Wilcoxon signed rank test respectively.

Based on the dimensions and algorithms, we reorder the above experimental data into vectors, which are suggested to represent the hypothesis distribution of different algorithms. Then we make the pairwise comparisons between the vector data of PPPA algorithm and other vectors one by one through the hypothesis testing on different dimensions. And thus we gain the probability of consistency (p-value of a two-sided Wilcoxon rank sum test and the signed rank test) and the result of significant differences testing. Meanwhile, a Bonferroni adjustment is also needed and a rigorous default significance level $p = 0.001$ is used in our experiment. If the p-value is less than 0.001, there are significant differences between two testing algorithms, or there are not.

Table 2 shows the runtime significant probability of consistency (p-value and results), and the Table 3 shows the corresponding error rates. As mentioned in above section, error rates were only generated in PPPA and FPRAS algorithm, so the statistical test of the error rates can only be applied on these two algorithms. Besides, our experiment does not have enough data for HBDA to conduct the analyses when $d \geq 13$, we mark them a '-' in Table 2. It can been seen that the proposed PPPA algorithm are significant difference with the other algorithms except when the QHV in 8D, HBDA in 8D and WFG in 10D. These cases mean that when $d = 8$, PPPA is not difference with the QHV and HBDA, and PPPA is not significantly better than WFG when $d = 10$.

Furthermore, combining these results with the experimental data of runtime, we can have the following conclusions.

Table 2
Significant differences analysis for PPPA in term of runtime (the probability of consistency (p-value) and the logical result of the hypothesis test), the failures of testing are highlighted in bold.

	Testing	5D	8D	10D	13D	15D
FPRAS	Rank sum	6.106E-37(1)	6.106E-37(1)	1.703E-36(1)	2.228E-09(1)	1.400E-26(1)
	Signed rank	1.868E-19(1)	1.868E-19(1)	1.868E-19(1)	1.715E-09(1)	5.362E-15(1)
QHV	Rank sum	5.781E-20(1)	**4.025E-01(0)**	2.573E-12(1)	2.632E-22(1)	6.795E-08(1)
	Signed rank	1.868E-19(1)	6.333E-05(1)	2.113E-17(1)	2.462E-14(1)	8.857E-05(1)
WFG	Rank sum	3.657E-27(1)	6.955E-08(1)	**4.506E-03(0)**	4.175E-30(1)	3.026E-12(1)
	Signed rank	1.868E-19(1)	1.186E-05(1)	9.327E-06(1)	5.697E-18(1)	5.388E-07(1)
HBDA	Rank sum	7.848E-23(1)	2.033E-03(1)	4.649E-14(1)	–	–
	Signed rank	1.868E-19(1)	**7.084E-01(0)**	1.515E-11(1)	–	–

1 Indicates a support of significant differences with PPPA algorithm at a 0.001 significance level.
0 Indicates a rejection of significant differences with PPPA algorithm at a 0.001 significance level.

Table 3
Significant differences analysis for PPPA in term of relative errors (the probability of consistency (p-value) and the logical result of the hypothesis test), the failures of testing are highlighted in bold.

	Testing	5D	8D	10D	13D	15D
FPRAS	Rank sum	8.000E-05(1)	**5.109E-01(0)**	1.261E-09(1)	8.185E-08(1)	6.921E-05(1)
	Signed rank	3.986E-05(1)	**7.006E-01(0)**	5.606E-06(1)	1.370E-05(1)	1.210E-04(1)

1 Indicates a support of significant differences with PPPA algorithm at a 0.001 significance level.
0 Indicates a rejection of significant differences with PPPA algorithm at a 0.001 significance level.

- When $d < 8$, the exact calculation algorithms (QHV, WFG and HBDA) outperform the proposed PPPA algorithm with a obvious difference and advantage. When $d = 8$ to $d = 10$, they all show a similar performance, and the exact calculation algorithms are gradually losing their superiorities with the dimension increases. when $d \geq 13$, the proposed PPPA algorithm outperforms the exact calculation algorithms with a obvious difference and advantage. Therefore, we strongly suggest that when $d \leq 8$, exact calculation algorithms are recommended (especially QHV, WFG and HBDA, considering they have an accurate result), and when $d \geq 10$, PPPA algorithm is recommended.
- As for FPRAS, there is a huge difference between FPRAS and PPPA for all dimensions in term of runtime, too. Table 2 shows that both the maximum and the median of p-value (2.23E-09 and 1.87E-19) are much less than the default significance level. Therefore, combining the results of experimental data (often ten times faster than FPRAS), we suppose the proposed PPPA algorithm outperforms FPRAS in runtime.
- On the other hand, in term of error rates, the differences of these two algorithms may much smaller, according to Table 3. Firstly, it shows that the 8D case of hypothesis testing has a very small difference between FPRAS and PPPA (the p-value of rank sum test and signed rank test are 0.511 and 0.701 respectively), so when $d = 8$, these two algorithms have a same magnitude of relative errors. Secondly, the gap of other dimensional results between two algorithms maybe not too significant. The maximum of p-value is 1.210E-04 and the median is 5.45E-05, which is very close to 0.001.
- However, despite of that, these two algorithms still show a significant difference on a whole (most of the results are less than 0.001). Therefore, combining the results of experimental data (the relative errors of PPPA are often better than FPRAS), we suppose the proposed PPPA algorithm is better than FPRAS in term of error rates.

6. Conclusion

In this paper, we have proposed a new method to approximate the hypervolume value, which can effectively decrease the running time of hypervolume indicator calculation in high dimensional objective space. The performance of the proposed approximation hypervolume calculation method has been verified in comparison with the classical algorithms on the PFs of a set of widely-used MaOPs test problems. We have integrated this calculation method into a representative SMS-EMOA algorithm. Experimental studies show the integrated algorithm for MaOPs are promising in terms of both performance and runtime.

Acknowledgment

This work was supported in part by the National Natural Science Foundation of China under Grants 61673121, 61672444 and 61272366, in part by the Projects of Science and Technology of Guangzhou under Grant 201804010352, and in part by the China Scholarship Council.

Appendix A. Integrate into hypervolume based evolutionary algorithms

We propose an incremental strategy and integrate it into S Metric Selection Evolutionary Multi-Objective Algorithm (SMS-EMOA) [5]. Then we compare it with other algorithms, which include NSGA–III [15], HypE [2], MOEA/D [46], IBEA [48] and other algorithms in the framework of SMS-EMOA (SMS-EMOA^{IWFG} and SMS-EMOA^{exQHV}).

As mentioned in Section 1 and Section 2, calculating hypervolumes measurement can be adopted to evaluate contribution in EMO algorithms by using Eq. (4). For instance, SMS-EMOA in jMetal [18] utilizes HSO to compute hypervolume contributions (called SMS-EMOA^{HSO}).

However, instead of Eq. (4), some strategies are proposed recently to determine the least contribution by other unique approaches, which are called incremental algorithms. It has been shown that these incremental algorithms have a more efficiency [14,40] than Eq. (4). Therefore, two currently fastest incremental algorithms are considered to be compared in our experiments, IWFG [14] and exQHV [39,40]. We apply them into the framework of SMS-EMOA, which denoted as SMS-EMOA^{IWFG} and SMS-EMOA^{exQHV} respectively, and take them to test the high-dimensional performance.

Obviously, the Algorithm 2 (PPPA) also can be applied to EMO algorithms through the Eq. (4). However, in this paper, We prefer to propose a incremental version for PPPA algorithm because of the efficiency. Similarly, the incremental PPPA algorithm which applied into the framework of SMS-EMOA was denoted as SMS-EMOA^{PPPA}.

The procedure detail of SMS-EMOA can be found in [5] that we are not going to make more introductions, here we just provide the incremental algorithm of PPPA which is used for determining the least hypervolume contribution solution. The procedure is shown in Algorithm 3, where K is the preset parameter donate the number of segmentation times, the

Algorithm 3 The procedure of determining the least hypervolume contribution solution which was applied to the SMS-EMOA^{PPPA} in MaOPs.

Input: $\mathcal{F}$, the population set of solutions; depth $K \in \mathbb{N}$;
Output: s, the solution that contributes the least to hypervolume;

1: $\mathcal{G} \leftarrow \{\mathcal{F}\}$, $Con \leftarrow \{0, \ldots, 0\}$, standardize the population and map it into $[0, 1]^d$;
2: **for** $k \leftarrow 1$ **to** K **do**
3: $P \leftarrow \mathcal{G}_1$;Find the pivot $p \in P$ and its original point $p^* \in \mathcal{F}$;
4: Divide the P according to Algorithm 1 and Fig. 2,which generate d subsets $Q_1, \ldots, Q_d$ and the rest of set P which is updated;
5: $Con[p^*] \leftarrow Con[p^*] + [HV(\{p\}, B) - PPPA(P \backslash p, B, \rho, 1)]$, $p^* \in \mathcal{F}$;
6: $\mathcal{G} \leftarrow \mathcal{G} \cup Q_1, \ldots, Q_d$, $\mathcal{G} \leftarrow \mathcal{G} \backslash \mathcal{G}_1$;
7: **end for**
8: **for all** P in $\mathcal{G}$ **do**
9: Produce $\mathcal{S}$ by P, $\mathcal{S}$ is the sampling set which is generated according to Algorithm 2 and Fig. 4;
10: **for all** a in P **do**
11: $\mathcal{S}' \leftarrow \{\xi \mid \xi \in \mathcal{S}, \xi \prec a, \xi \not\prec b, b \in P, b \neq a\}$;
12: $Con[a^*] \leftarrow Con[a^*] + \left[HV(\{a\}, B) - \frac{|\mathcal{S}'|}{\rho}\right]$, $a^* \in \mathcal{F}$ is the original point of a;
13: **end for**
14: **end for**
15: **return** $s \leftarrow \underset{a \in \mathcal{F}}{\operatorname{argmin}} Con[a]$;

parameter of PPPA algorithm in line 5 is the same as Algorithm 2, $p^* \in \mathcal{F}$ is the original point of p, which is projected to the subsets and generates the projection p.

The process of Algorithm 3 is very similar to Algorithm 2, which also consists of two parts: the segmentation part and the sampling part. The most difference of these two algorithm is that in the segmentation part Algorithm 3 needs to calculate the contributions of the pivot points through Algorithm 2, but in Algorithm 2 it just computes a series of multiplications for the pivot points and has no need to calculate the contributions. Another difference is that Algorithm 3 records the sampling points and the original points of each solution which in a subset.

Different from calculating the hypervolume measurement, determining the least hypervolume contribution is more difficult, which often means that the hypervolume measurement has to be calculated N times if we want to find the one having the least contribution in N solutions. As is known, hypervolume approximation method would degrade the solving performance, but as well as improve calculation speed of each generation. Therefore, our purpose to design this version is to decrease the runtime of hypervolume based evolutionary algorithms and make possible to solve the MaOPs in the high dimension, meanwhile, the SMS-EMOA^{PPPA} algorithm abides the principles of speed first, performance is second.

A.1. Experimental settings

Experimental settings are similar to [15,45], but we modify some details in order to adapt the experimental algorithms, meanwhile, part of data are also from Yuan et al. [45].

As a basis for the comparisons, two well-known test suites for many-objective optimization, Deb-Thiele-Laumanns-Zitzler (DTLZ) [17] and Walking-Fish-Group (WFG) [25], are involved in the experiments.

We consider the number of objectives $d \in \{3, 5, 8, 10, 15\}$. For DTLZ 1–4 problems, the total number of decision variables is given by $m = d + h - 1$, unless otherwise specified, h is set to 5 for DTLZ 1, 10 for DTLZ 2–4 and 20 for DLTZ7 as recommended in [15]. As for all WFG problems, unless otherwise stated, the number of decision variables is set to 24 and the position-related parameter is set to $d - 1$ according to [25].

These MOPs have a variety of characteristics, such as having linear, mixed (convex/concave), multi-modal, disconnected, degenerate, and disparately scaled PFs, which challenge different abilities of an algorithm.

The experimental settings include general settings and parameter settings. The general settings are listed as the follows.

1. Number of runs: Each algorithm is run 20 times independently for each test instance;
2. Termination criterion: Generally, there are two kinds of termination criterion for each run, the maximum number of generations (MaxGen) and the maximum number of function evaluations (MaxFES). Since the compared algorithms are of varying computational complexity, we mainly use different MaxFES for different problems (instances). The settings detail of MaxFES is correspondingly shown in Table A.2.

As for the parameter settings, several common settings for algorithms are first given as follows

1. Population size: The setting of population size N for NSGA–III and MOEA/D cannot be arbitrarily specified, so we follow the settings of Yuan et al. [45] in this study, which means the population size N is set to $N = \{91, 210, 156, 275, 135\}$ for $d = \{3, 5, 8, 10, 15\}$ objectives, respectively.
2. Parameters for crossover and mutation: The simulated binary crossover (SBX) and polynomial mutation [45] are used in all the considered algorithms. The crossover probability (p_c) and mutation probability (p_m) are respectively set to $p_c = 1.0$ and $p_m = 1/n$, distribution index for both crossover (η_c) and mutation (η_m) are set to 20.
3. Specific parameters of algorithms: The neighborhood size T in MOEA/D is set to 20. In HypE, The bound of the reference point is set to 200, and the number of sampling points M is set to 10,000. The fitness scaling factor in IBEA is set to 0.05. The depth K in the proposed algorithm is set to 5.

A.2. Experimental results and discussion

First of all, We record the runtime of each generation, $d \in \{3, 5, 8, 10, 15\}$. As mentioned above, the bottleneck of hypervolume based algorithms on solving MaOPs is the high runtime complexity, so the runtime of the proposed algorithm is primarily concerned. Table A.1 shows the comparison data of runtime and these data are an average which are obtained by collecting the runtime of each generation and removing the extreme data (maximum data, minimum data and some irrational data produced by manual mistakes).

As seen from the Table A.1, the runtime of proposed algorithm is less than satisfactory and generally slower than other algorithms, when $d \leq 5$. Meanwhile, the IWFG method is always faster than the exQHV method, IWFG method could determine the least contribution solution without N times calculations. In the best case, IWFG would even just calculate the hypervolume measurement once instead of N times. However, its advantage would be smaller with the increase of dimensions. When $d \geq 8$, the proposed algorithm obtains the least runtime. Moreover, it is tens or hundreds of times faster than SMS-EMOA^{IWFG} and SMS-EMOA^{exQHV} after $d > 10$.

The runtime of SMS-EMOA^{PPPA} keeps within 15 s, in particular, when $d = 15$, it takes about 6 s for SMS-EMOA^{PPPA} in DTLZ 1, because there is less time spent of the proposed algorithm in calculating this instance type (linear instances). Unfortunately, SMS-EMOA^{IWFG} could not get faster runtime after $d \geq 10$ which generally takes nearly 100 s. From the perspectives of time consumption, the runtime of SMS-EMOA^{PPPA} in calculating each generation is acceptable even $d = 15$, because it takes reasonable and acceptable time to calculate MaOPs completely (often less than one day). By comparison, SMS-EMOA^{IWFG} and SMS-EMOA^{exQHV} take months to complete the MaOPs calculation, which is not accepted, and hence we do not show their experimental results in this paper (we mark a '-' in tables).

Table A.2 presents the performance of every algorithm. The hypervolume-indicator is used to evaluate the performance of the concerned algorithms in our experiments. the calculation boundary is set as $B = (1.1, \ldots, 1.1)$ for DTLZ problems, and $B = (2.1, 4.1, \ldots, 2d + 0.1)$ for WFG problems. When dimension is greater than 10, we approximate the hypervolume measure by the Monte Carlo simulation proposed in [2], and 10,000,000 sampling points are also used. To facilitate a better observation of the population, we also plot the final solutions set of our algorithm on the 15-objective DTLZ and WFG instances in a single run by parallel coordinates in Fig. A.1. This particular run is associated with the result closest to the average hypervolume value, which means that the SMS-EMOA^{PPPA} algorithm has able to find a good approximation and coverage of the PF, according to [17] and [25].

The bold number is the best quality indicator in a row for calculating the same MaOPs. From Table A.2, SMS-EMOA^{IWFG}, SMS-EMOA^{exQHV} have similar performance and both of them have best quality indicators because of the benefits of exact contributions calculation when $d \leq 8$. By contrast, SMS-EMOA^{PPPA} is designed by a scheme which gains the least contribution approximately, so the approximation scheme may impact the performances in the same MaxFES and fail to get a best quality indicator in the low dimensions. Nevertheless, this algorithm also has a good performance and the hypervolume-indicator of SMS-EMOA^{PPPA} is general better than NSGA–III and MOEA/D.

Table A.1
Time median of one generation by SMS-EMOA^{PPPA} and other EMO algorithms on various MOPs (s).

EMO algorithms	m	DTLZ1	DTLZ2	DTLZ3	DTLZ4
SMS-EMOA^{PPPA}	3	5.594924e-01	6.557407e-01	1.057407e+00	6.571168e-01
	5	8.147231e-01	1.139480e+00	1.269868e+00	1.033759e+00
	8	2.217613e+00	2.617139e+00	3.713178e+00	2.234578e+00
	10	3.468815e+00	5.075068e+00	7.648885e+00	4.576131e+00
	15	6.157355e+00	8.594924e+00	1.147237e+01	9.057919e+00
SMS-EMOA^{IWFG}	3	1.648885e-01	1.176131e-01	1.853756e-01	1.402805e-01
	5	2.093648e+00	2.760251e+00	3.189977e+00	2.983641e+00
	8	2.512671e+01	2.550951e+01	3.059571e+01	2.990767e+01
	10	1.386244e+02	1.472155e+02	1.575083e+02	1.499838e+02
	15	–	–	–	–
SMS-EMOA^{exQHV}	3	7.157355e-01	6.443181e-01	8.115805e-01	7.328256e-01
	5	7.537291e+00	8.356987e+00	1.087198e+01	8.948961e+00
	8	9.340107e+01	9.630885e+01	9.879820e+01	8.851680e+01
	10	–	–	–	–
	15	–	–	–	–
EMO algorithms	m	WFG5	WFG6	WFG7	WFG8
SMS-EMOA^{PPPA}	3	9.691293e-01	9.339932e-01	1.183285e+00	8.769230e-01
	5	1.323592e+00	1.540409e+00	1.468815e+00	1.575068e+00
	8	3.948286e+00	3.815585e+00	4.455862e+00	2.863130e+00
	10	6.705928e+00	7.571669e+00	9.784982e+00	6.853756e+00
	15	1.469868e+01	1.323592e+01	1.692207e+01	9.754040e+00
SMS-EMOA^{IWFG}	3	2.418863e-01	2.217613e-01	2.571669e-01	2.059287e-01
	5	3.597440e+00	3.403857e+00	3.852678e+00	3.238119e+00
	8	2.909033e+01	3.472155e+01	3.592914e+01	2.802155e+01
	10	1.510839e+02	1.572002e+02	1.732888e+02	1.308286e+02
	15	–	–	–	–
SMS-EMOA^{exQHV}	3	9.390016e-01	1.077423e+00	1.101246e+00	8.443088e-01
	5	1.028431e+01	8.551580e+00	1.078583e+01	7.303309e+00
	8	9.132868e+01	9.786806e+01	1.047222e+02	8.865119e+01
	10	–	–	–	–
	15	–	–	–	–

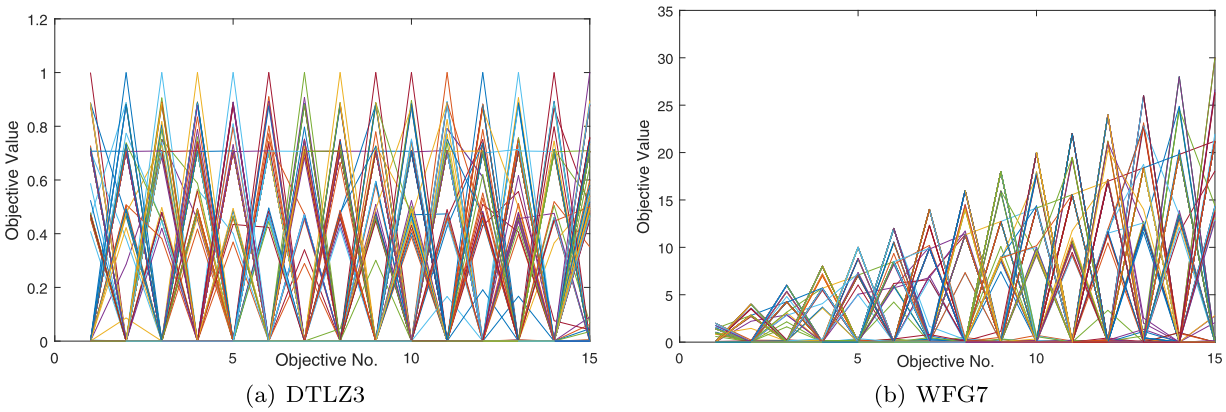


Fig. A.1. Parallel coordinate plots on the 15-objective DTLZ3 and WFG7 instances.

As mentioned above, when $d \geq 10$, the runtime of SMS-EMOA^{exQHV} is beyond the scope of time that we can wait, which causes that it can not be evaluated the performance (we mark a '-' in tables). A similar situation also happeneds in the case of SMS-EMOA^{IWFG} when $d = 15$. Therefore, though these two algorithms have a best quality indicator in low dimensions, they can not continue these performances to high dimensions (often stop at 8-dimensional space).

Besides, after $d \geq 10$, the proposed SMS-EMOA^{PPPA} becomes the best algorithm, which obtains best quality indicators in both DTLZ and WFG problems.

From Table A.2, SMS-EMOA^{PPPA} obtains four best quality indicators in 10-dimension and five best in 15-dimension of eight instances, much more than other algorithms. Moreover, the performance of SMS-EMOA^{PPPA} is also satisfactory and very close to best quality for all testing problems.

Although the proposed algorithm is an approximation process when selecting offsprings, the experimental result of this process is very close to that of the accuracy process (like IWFG and exQHV), it is difficult to differentiate them through the

Table A.2
Performance comparison of the proposed algorithm to different algorithms with respect to the average HV values on DTLZ and WFG problems, the best quality indicators in a row are highlighted.

Problem	Obj	MaxFES ($\times 10^3$)	SMS^{PPPA}	NSGA–III	MOEA/D	IBEA	HypE	SMS^{IWFG}	SMS^{exQHV}
DTLZ1	3	37	1.272376	1.117600	1.116679	1.257326	1.117483	1.299868	**1.304145**
	5	126	1.595686	1.577027	1.577632	1.600609	1.466936	1.603491	**1.608678**
	8	117	2.130458	2.137837	2.136337	2.142805	1.999087	**2.143586**	2.141192
	10	275	**2.593743**	2.592792	2.592233	2.593646	2.520526	2.593040	–
	15	203	**4.177216**	4.176773	4.169859	4.176120	3.702937	–	–
DTLZ2	3	23	0.751386	0.743523	0.744137	0.740826	**0.753191**	0.751337	0.742359
	5	74	1.316218	1.303638	1.307343	1.301332	0.756079	**1.324135**	1.321852
	8	78	1.956183	1.969096	1.978216	**2.001752**	0.892917	1.987125	2.000517
	10	207	2.561711	2.508717	2.515040	2.526253	1.174930	**2.563844**	–
	15	135	**4.157259**	4.133743	4.137792	4.143877	1.407601	–	–
DTLZ3	3	91	**0.750386**	0.737407	0.736044	0.329928	0.750325	0.749683	0.749187
	5	210	1.305619	1.301481	1.303168	0.619372	0.740621	1.317255	**1.323951**
	8	156	1.921278	1.954336	1.251873	1.176334	0.881516	1.914549	**1.930525**
	10	413	2.563302	2.508879	2.406221	1.619347	1.175350	**2.563669**	–
	15	270	**4.157025**	4.123622	2.722371	2.273451	1.553939	–	–
DTLZ4	3	55	0.462418	0.744634	0.406020	0.742538	0.549999	0.747674	**0.752728**
	5	210	1.323590	1.308698	1.205512	1.304583	1.014145	**1.324306**	1.320124
	8	195	1.908975	1.980236	1.826489	2.007174	0.925370	**2.043475**	1.981841
	10	550	2.523918	2.515172	2.423727	**2.529864**	1.235517	2.521993	–
	15	405	4.117135	**4.138154**	3.978200	4.131603	2.056801	–	–
WFG5	3	37	0.698515	0.687220	0.656189	0.513921	**0.698642**	0.687551	0.698317
	5	158	1.222758	1.222480	1.120619	0.749296	0.893813	**1.223174**	1.222384
	8	234	1.749273	**1.850281**	1.279934	0.858949	1.183477	1.725009	1.725158
	10	550	2.245601	**2.346581**	1.541144	0.904139	1.659310	2.243598	–
	15	405	3.790247	**3.833242**	1.864379	1.586927	2.764870	–	–
WFG6	3	37	0.705901	0.685939	0.654956	0.530575	0.708633	0.709415	**0.709427**
	5	158	1.003956	1.219001	1.041593	0.769356	0.441287	1.220474	**1.221972**
	8	234	1.450689	1.843340	0.698152	0.883875	0.475371	**1.853215**	1.805814
	10	550	**2.343985**	2.326666	0.811370	0.912926	0.627106	2.331689	–
	15	405	3.480872	**3.717982**	0.594620	2.022430	1.123090	–	–
WFG7	3	37	0.748612	0.729030	0.619351	0.553872	0.752768	**0.757109**	0.756832
	5	158	1.218317	**1.291999**	1.073783	0.802963	0.525322	1.229043	1.224217
	8	234	1.977260	1.971529	0.813288	0.925286	0.521515	1.971560	**1.990125**
	10	550	**2.558091**	2.507511	0.950840	0.971469	1.234357	2.557977	–
	15	405	**4.152862**	4.134418	0.772304	2.479611	2.267561	–	–
WFG8	3	37	0.685801	0.665932	0.633207	0.470482	**0.686264**	0.671924	0.675490
	5	158	1.160157	1.182260	0.968246	0.679012	0.340591	**1.183049**	1.213823
	8	234	1.773044	1.759882	0.326124	0.815046	0.693903	**1.800519**	1.782945
	10	550	**2.297019**	2.280276	0.255629	0.948167	1.014398	2.028579	–
	15	405	**3.880247**	3.815520	0.706627	1.500565	1.596445	–	–

SMS^{PPPA} indicates the SMS-EMOA^{PPPA} algorithm.
SMS^{IWFG} indicates the SMS-EMOA^{IWFG} algorithm.
SMS^{exQHV} indicates the SMS-EMOA^{exQHV} algorithm.

results of Table A.2. The impact of errors which produced by the approximation process in high dimensions is very small, and we suggest it can be ignored.

In this comparison, NSGA–III and MOEA/D did not obtain the best quality indicator because we use hypervolume-indicator as the quality indicator. Despite the aforementioned advantages of the hypervolume indicator, it inevitably has its biases. Meanwhile, the proposed algorithm is based on this indicator, and the solutions is also with this biases. That is why the proposed algorithm has advantages in the hypervolume-indicator evaluation. NSGA–III and MOEA/D based on other methods are not having this biases, which leads that they have no best indicator in hypervolume-indicator measurement.

HypE is a hypervolume based evolutionary algorithm, however, its performance is bad and it does not gain a best quality indicator. IBEA is an indicator-based EMO algorithm that is similar to NSGA–II except a diversity preservation scheme. From Table A.2, we can see that IBEA obtains the best quality indicator twice for DTLZ problems, and it is also very close to the best in other DTLZ tests. It suggests that it is perfectly desirable for the DTLZ problems. Unfortunately, it can not extend to the WFG problems. When solving WFG problems, IBEA has a worst performance in all testing algorithms. So it means that IBEA is a bias algorithm that performs well on some problems but not so well on the others. Its robustness is not as good as SMS-EMOA^{PPPA}.

In summary, considering the comparison of time and performance, we believe the proposed algorithm makes a breakthrough for the hypervolume based evolutionary algorithms in many-objective optimization problems.

References

[1] J. Bader, K. Deb, E. Zitzler, Faster hypervolume-based search using monte carlo sampling, Lect. Notes Econ. Math. Syst. 634 (32) (2010) 313–326.

[2] J. Bader, E. Zitzler, Hype: an algorithm for fast hypervolume-based many-objective optimization, Evol. Comput. 19 (1) (2011) 45–76.
[3] N. Beume, S-metric calculation by considering dominated hypervolume as Klee's measure problem, Evol. Comput. 17 (4) (Dec. 2009) 477–492.
[4] N. Beume, C.M. Fonseca, J. Vahrenhold, On the complexity of computing the hypervolume indicator, IEEE Trans. Evol. Comput. 13 (5) (2009) 1075–1082.
[5] N. Beume, B. Naujoks, SMS-EMOA: multiobjective selection based on dominated hypervolume, Eur. J. Oper. Res. 181 (3) (Feb. 2007) 1653–1669.
[6] L. Bradstreet, L. While, L. Barone, A fast incremental hypervolume algorithm, IEEE Trans. Evol. Comput. 12 (6) (2008) 714–723.
[7] K. Bringmann, T. Friedrich, Approximating the volume of unions and intersections of high-dimensional geometric objects, Int. Symp. Algorithms Comput. 43 (6) (2008) 436–447.
[8] K. Bringmann, T. Friedrich, Parameterized average-case complexity of the hypervolume indicator, in: Conference on Genetic and Evolutionary Computation., 2013, pp. 575–582.
[9] D. Brockhoff, J. Bader, L. Thiele, E. Zitzler, Directed multiobjective optimization based on the weighted hypervolume indicator, J. Multi-Criteria Decis. Anal. 20 (5–6) (2013) 291–317.
[10] D. Brockhoff, T. Friedrich, F. Neumann, Analyzing hypervolume indicator based algorithms, in: International Conference on Parallel Problem Solving From Nature: PPSN X, vol. 5199, 2008, pp. 651–660.
[11] D. Brockhoff, T. Friedrich, et al., On the effects of adding objectives to plateau functions, IEEE Trans. Evol. Comput. 13 (3) (Jun. 2009) 591–603.
[12] L. Chen, H. Liu, K. Tan, Y. Cheung, Y. Wang, Evolutionary many-objective algorithm using decomposition based dominance relationship, IEEE Trans. Cybern. (2017), doi:10.1109/TCYB.2018.285917.
[13] Y. Cheung, F. Gu, H. Liu, Objective extraction for many-objective optimization problems: algorithm and test problems, IEEE Trans. Evol. Comput. 20 (5) (2016) 755–772.
[14] W. Cox, L. While, Improving the IWFG algorithm for calculating incremental hypervolume, in: IEEE Congress on Evolutionary Computation IEEE (CEC), 2016, pp. 3969–3976.
[15] K. Deb, H. Jain, An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part i: solving problems with box constraints, IEEE Trans. Evol. Comput. 18 (4) (2014) 577–601.
[16] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and flitist multiobjective genetic algorithm: NSGA–II, IEEE Trans. Evol. Comput. 6 (2) (2002) 182–197.
[17] K. Deb, L. Thiele, M. Laumanns, Z. E, Scalable multi-objective optimization test problems, Congr. Evol. Comput. (CEC) 1 (2002) 825–830.
[18] J.J. Durillo, A.J. Nebro, jMetal: a java framework for multi-objective optimization, Adv. Eng. Softw. 42 (10) (2011) 760–771.
[19] M. Fleischer, "The measure of pareto optima: applications to multiobjective metaheuristics", Int. Conf. Evol. Multi-Criterion Optim. 2632 (1) (2003) 519–533.
[20] T. Friedrich, C. Horoba, F. Neumann, Multiplicative approximations and the hypervolume indicator, Conf. Genet. Evol. Comput. ACM (2009) 571–578.
[21] I.C. García, C.A.C. Coello, A. Arias-Montaño, MOPSOhv: a new hypervolume-based multi-objective particle swarm optimizer, in: IEEE Congress on Evolutionary Computation (CEC), 2014, pp. 266–273.
[22] F. Gu, Y. Cheung, Self-organizing map-based weight design for decomposition-based many-objective evolutionary algorithm, IEEE Trans. Evol. Comput. 22 (2) (2018) 211–225.
[23] F. Gu, H. Liu, Y. Cheung, S. Xie, Optimizing wcdma network planning by multiobjective evolutionary algorithm with problem-specific genetic operation, Knowl. Inf. Syst. (KAIS) 45 (3) (2015) 679–703.
[24] P. Guerreiro, C.M. Fonseca, M.T. Emmerich, A fast dimension-sweep algorithm for the hypervolume indicator in four dimensions, in: Canadian Conference on Computational Geometry (CCCG 2012), 2012, pp. 77–82.
[25] S. Huband, P. Hingston, L. Barone, L. While, A review of multiobjective test problems and a scalable test problem toolkit, IEEE Trans. Evol. Comput. 10 (5) (2006) 477–506.
[26] C. Igel, N. Hansen, S. Roth, Covariance matrix adaptation for multi-objective optimization, Evol. Comput. 15 (1) (2007) 1.
[27] S. Jiang, J. Zhang, Y.S. Ong, et al., A simple and fast hypervolume indicator-based multiobjective evolutionary algorithm, IEEE Trans. Cybern. 45 (10) (2015) 2202–2213.
[28] R. Lacour, K. Klamroth, C.M. Fonseca, A box decomposition algorithm to compute the hypervolume indicator, Comput. Oper. Res. 79 (5) (Mar, 2017) 347–360.
[29] B. Li, K. Tang, J. Li, X. Yao, Stochastic ranking algorithm for many-objective optimization based on multiple indicators, IEEE Trans. Evol. Comput. 20 (2016) 924–938, doi:10.1109/TEVC.2016.2549267.
[30] H. Liu, F. Gu, Y. Cheung, T-MOEA/D: MOEA/D with objective transform in multi-objective problems, in: Proceedings of 2010 International Conference on Information Science and Management Engineering, 2010, pp. 282–285.
[31] H. Liu, F. Gu, Q. Zhang, Decomposition of a multiobjective optimization problem into a number of simple multiobjective subproblems, IEEE Trans. Evol. Comput. 18 (3) (Jun. 2014) 450–455.
[32] A. Menchaca-Mendez, C.A.C. Coello, A new selection mechanism based on hypervolume and its locality property, in: 2013 IEEE Congress on Evolutionary Computation (CEC), Jul. 2013, pp. 924–931, doi:10.1109/CEC.2013.6557666.
[33] K. Narukawa, T. Rodemann, Examining the performance of evolutionary many-objective optimization algorithms on a real-world application, in: International Conference on Genetic and Evolutionary Computing IEEE, 2013, pp. 316–319.
[34] B. Naujoks, N. Beume, M. Emmerich, Multi-objective optimisation using s-metric selection: application to three-dimensional solution spaces, in: IEEE Congress on Evolutionary Computation (CEC), vol. 2, 2005, pp. 1282–1289.
[35] K. Nowak, M. Märtens, D. Izzo, "Empirical performance of the approximation of the least hypervolume contributor", in: International Conference on Parallel Problem Solving From Nature, vol. 8672, 2014, pp. 662–671.
[36] M.H. Overmars, C.K. Yap, New upper bounds in Klee's measure problem, in: Proceedings of Annual Symposium Foundations of Computer Science (FOCS), vol. 20, 2006, pp. 550–556.
[37] C. Priester, K. Narukawa, T. Rodemann, A comparison of different algorithms for the calculation of dominated hypervolumes, in: Genetic and Evolutionary Computation Conference (GECCO), vol. 21, 2013, pp. 655–662.
[38] S. Rostami, F. Neri, A fast hypervolume driven selection mechanism for many-objective optimisation problems, Swarm Evol. Comput. 34 (5) (2017) 50–67.
[39] L.M.S. Russo, A.P. Francisco, "Quick hypervolume", IEEE Trans. Evol. Comput. 18 (4) (2014) 481–502.
[40] L.M.S. Russo, A.P. Francisco, "Extending quick hypervolume", J. Heuristics 22 (3) (2016) 245–271.
[41] O. Schutze, A. Lara, C.A.C. Coello, On the influence of the number of objectives on the hardness of a multiobjective optimization problem, IEEE Trans. Evol. Comput. 15 (4) (Aug. 2011) 444–455.
[42] H. Trautmann, T. Wagner, D. Brockhoff, R2-EMOA: focused multiobjective search using R2-indicator-based selection, in: Learning and Intelligent Optimization, vol. 7997, Springer Berlin Heidelberg, 2013, pp. 70–74.
[43] L. While, L. Bradstreet, Applying the WFG algorithm to calculate incremental hypervolumes, Evol. Comput. IEEE 22 (10) (2012) 1–8.
[44] L. While, L. Bradstreet, L. Barone, A fast way of calculating exact hypervolumes, IEEE Trans. Evol. Comput. 16 (1) (2012) 86–95.
[45] Y. Yuan, H. Xu, B. Wang, X. Yao, A new dominance relation based evolutionary algorithm for many-objective optimization, IEEE Trans. Evol. Comput. 20 (1) (2016) 16–37.
[46] Q. Zhang, H. Li, MOEA/D: a multiobjective evolutionary algorithm based on decomposition, IEEE Trans. Evol. Comput. 11 (6) (2007) 712–731.
[47] E. Zitzler, D. Brockhoff, L. Thiele, The hypervolume indicator revisited: on the design of pareto-compliant indicators via weighted integration, in: International Conference on Evolutionary Multi-Criterion Optimization, 4403, 2007, pp. 862–876.
[48] E. Zitzler, S. Künzli, Indicator-based selection in multiobjective search, in: Parallel Problem Solving from Nature-PPSN VIII, Aug. 2004, pp. 832–842.
[49] E. Zitzler, L. Thiele, Multiobjective optimization using evolutionary algorithms-a comparative case study, in: Parallel Problem Solving from Nature-PPSN V, Springer Berlin Heidelberg, 1998, pp. 292–301.